HERO VIRED | DEVOPS PROGRAM | ASSIGNMENT REPORT

Deploying a MERN Application on AWS using Terraform and Ansible

Implementation report with screenshots: TravelMemory (MongoDB, Express, React, Node.js)

My git repo → https://github.com/dhanacodes24/mern-terraform-ansible-assignment.git

Field	Details (fill in)
Student name	Dhanashree
Program / batch	Hero Vired DevOps Program, [B]
Assignment	Deploying a MERN Application through Terraform and Ansible
Application	TravelMemory: github.com/UnpredictablePrashant/TravelMemory
Cloud region	AWS US East (N. Virginia), `us-east-1`
Submission date	[09 Nov 2026]
GitHub repository	https://github.com/dhanacodes24/mern-terraform-ansible-assignment.git
Evidence format	Screenshots

Contents

1. Introduction and objective
2. Architecture overview and environment details
3. Tools and prerequisites
4. Part 1: Infrastructure setup with Terraform (Tasks 1 to 5)
5. Part 2: Configuration and deployment with Ansible (Tasks 1 to 5)
6. How the application components interact
7. Working application: proof of deployment
8. Security summary
9. Idempotency, cost control and clean-up
10. Conclusion and future improvements
11. Deliverables checklist and repository details

1. Introduction and objective

1.1 Objective

The objective of this assignment is to deploy the TravelMemory MERN application on AWS without manual console work. **Terraform** creates the cloud infrastructure (network, servers, security and IAM) and **Ansible** configures the servers and deploys the application on them. Repeating the commands produces the same result each time.

1.2 Application used

TravelMemory is a travel diary application. It has a **React** frontend where users add and view trip experiences, an **Express/Node.js** backend that exposes a REST API (`/trip`), and a **MongoDB** database that stores the trips.

1.3 Scope covered

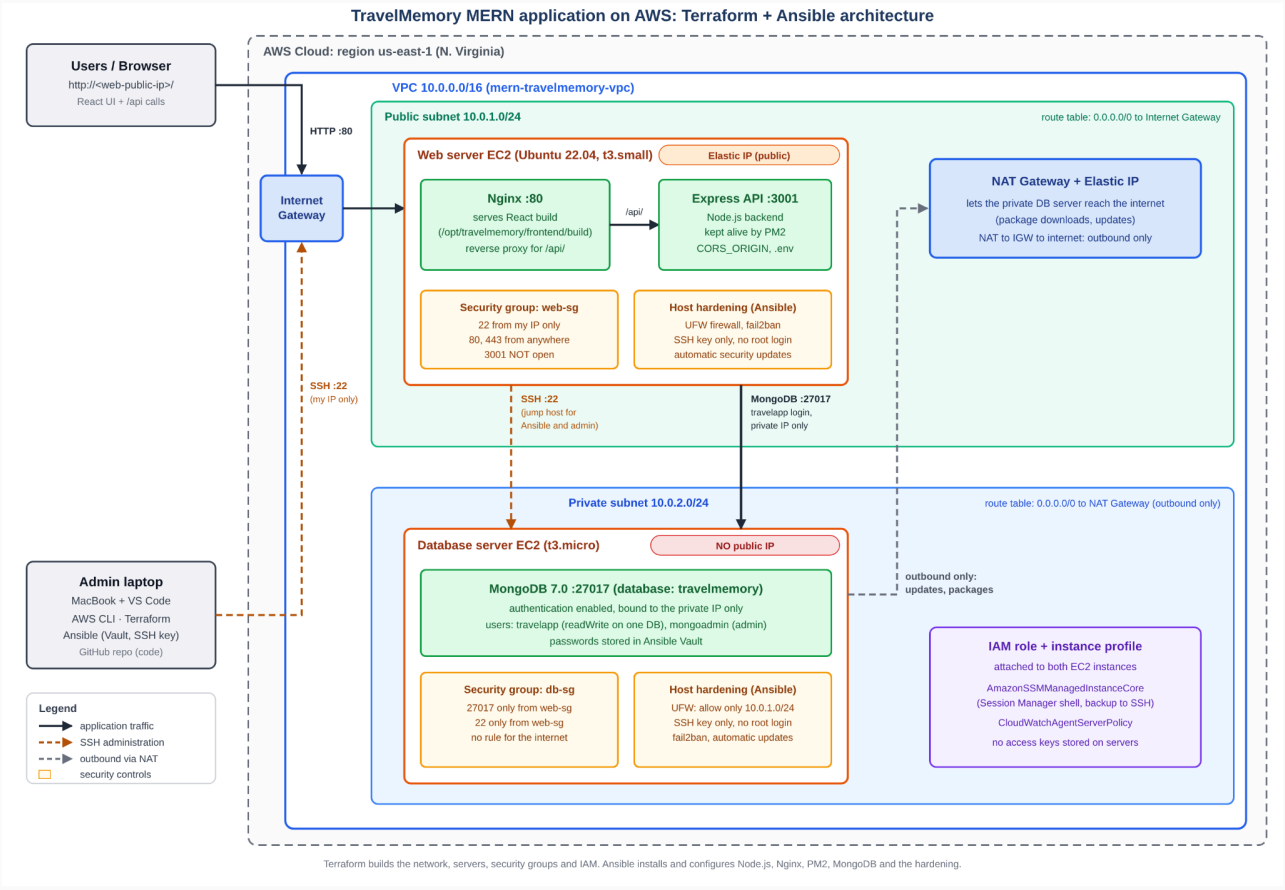
Assignment part	What was implemented
Part 1: Terraform	AWS CLI authentication, Terraform project, VPC with public and private subnets, Internet Gateway, NAT Gateway, route tables, two EC2 instances, security groups, IAM role and instance profile, output of the web server public IP
Part 2: Ansible	Ansible configuration with a jump host, Vault for secrets, Node.js/NPM and Nginx installation, repository clone and dependencies, MongoDB installation with authentication, users and database, environment variables, PM2, React to Express connection, firewall (UFW), SSH hardening and fail2ban

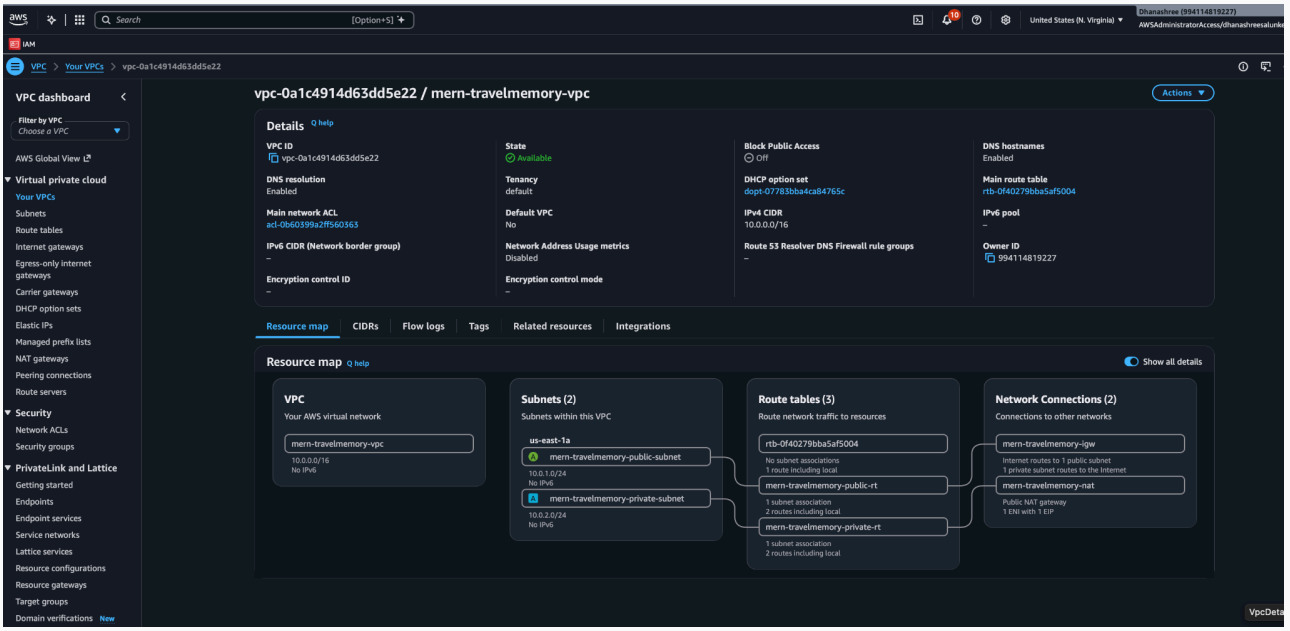
2. Architecture overview and environment details

2.1 Architecture diagram

The web server sits in the public subnet and the database server sits in the private subnet. Only the web server is reachable from the internet. The database accepts connections only from the web server's security group.

The Architecture diagram and VPC resource map from the AWS console (VPC > select your VPC > Resource map tab)





pasting your image)

Figure 1: *Architecture and VPC, public and private subnets, gateways and servers*

2.2 Request flow

- The user opens `http://<web-public-ip>/`. Traffic enters through the **Internet Gateway** and the web server's **security group** (port 80).
- **Nginx** serves the React build files. React runs in the user's browser.
- The browser calls `http://<web-public-ip>/api/...`. Nginx forwards `/api/` to the **Express** backend on `127.0.0.1:3001`.
- Express connects to **MongoDB** on the database server's private IP, port 27017, using the `travelapp` user.
- The database server has no public IP. It downloads packages through the **NAT Gateway** (outbound only).

3. Tools and prerequisites

The work was done on a MacBook using Visual Studio Code and the macOS terminal. The following tools were installed and verified.

Terminal output of: terraform -version, ansible --version, aws --version.

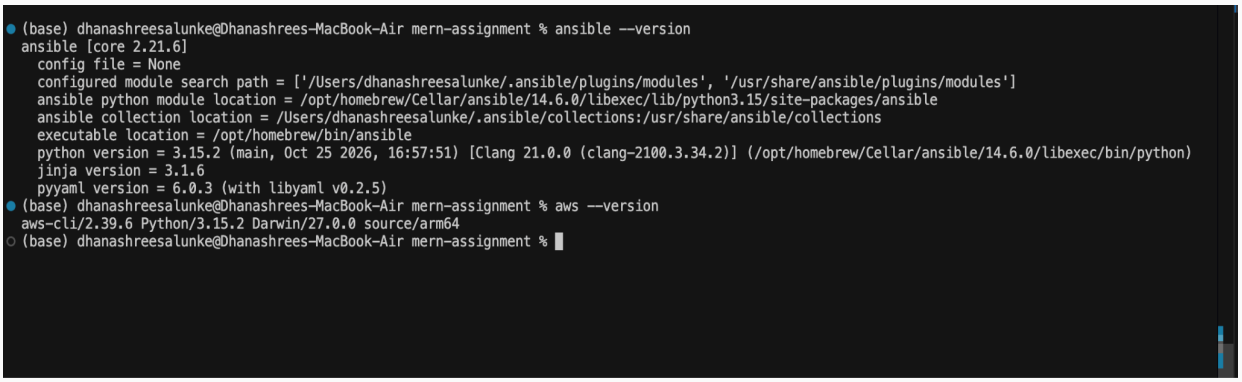


Figure 2: *Installed tool versions*

Code with the project folder open, showing the terraform/ and ansible/ structure in the Explorer panel.

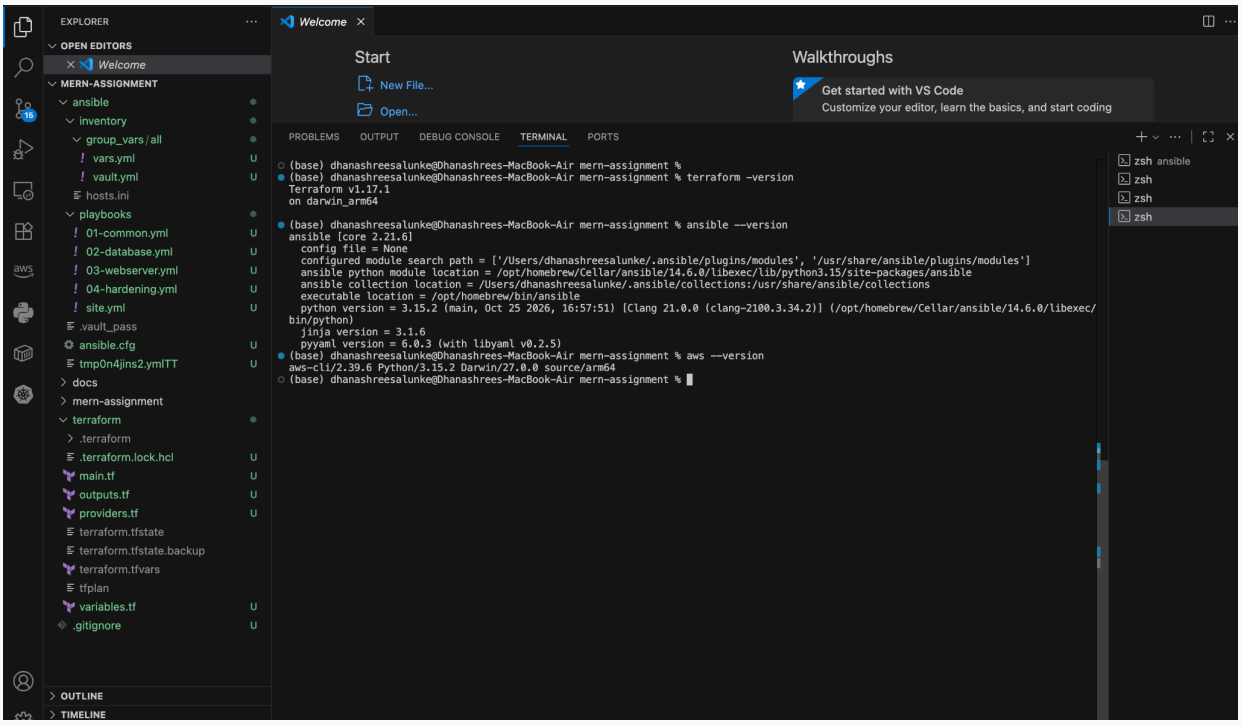


Figure 3: *Project structure in Visual Studio Code*

3.1 SSH key pair

An ed25519 key pair was created. The public key is installed on both EC2 instances by Terraform; the private key stays on the laptop with permission 600.

Terminal output of: ls -l ~/.ssh/mern-key ~/.ssh/mern-key.pub (private key must show -rw-------). Do NOT show the key contents.

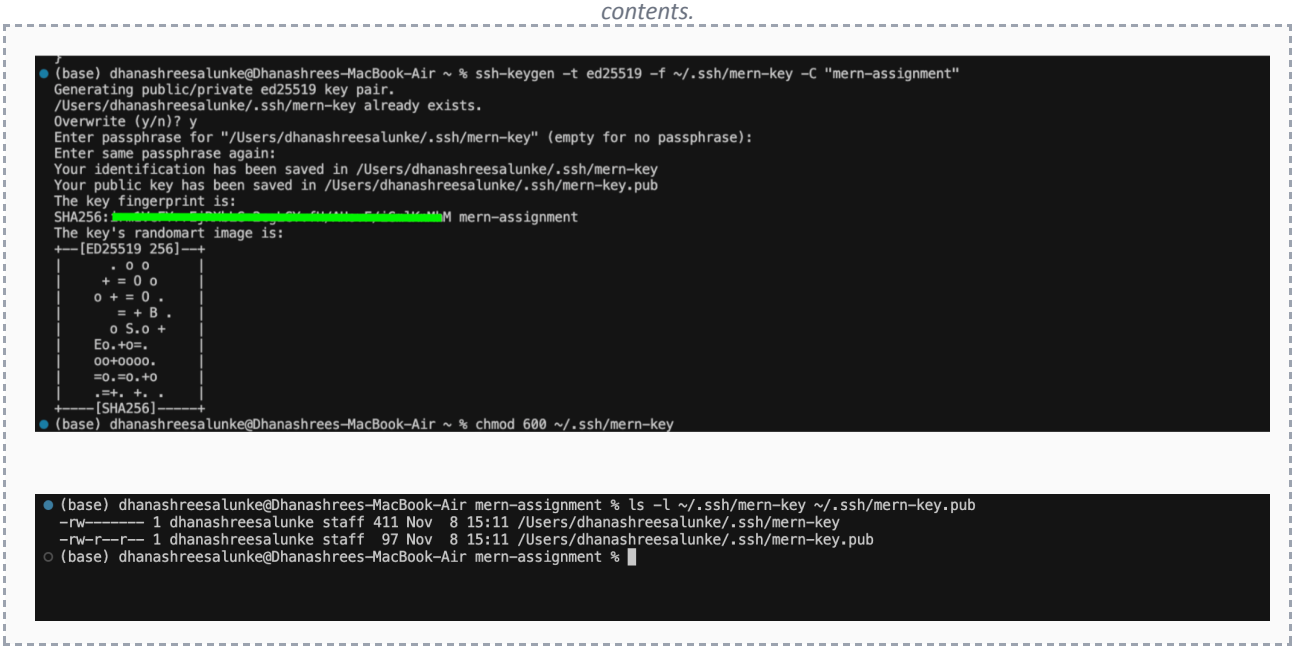


Figure 4: *SSH key pair created with correct permissions*

4. Part 1: Infrastructure setup with Terraform

4.1 Task 1: AWS setup and Terraform initialisation

A dedicated IAM user (`terraform-student`) was created for the AWS CLI, and the CLI was configured with its access key. Terraform was initialised with the AWS provider in the `terraform/` folder.

IAM console > Users > terraform-student (Permissions tab)

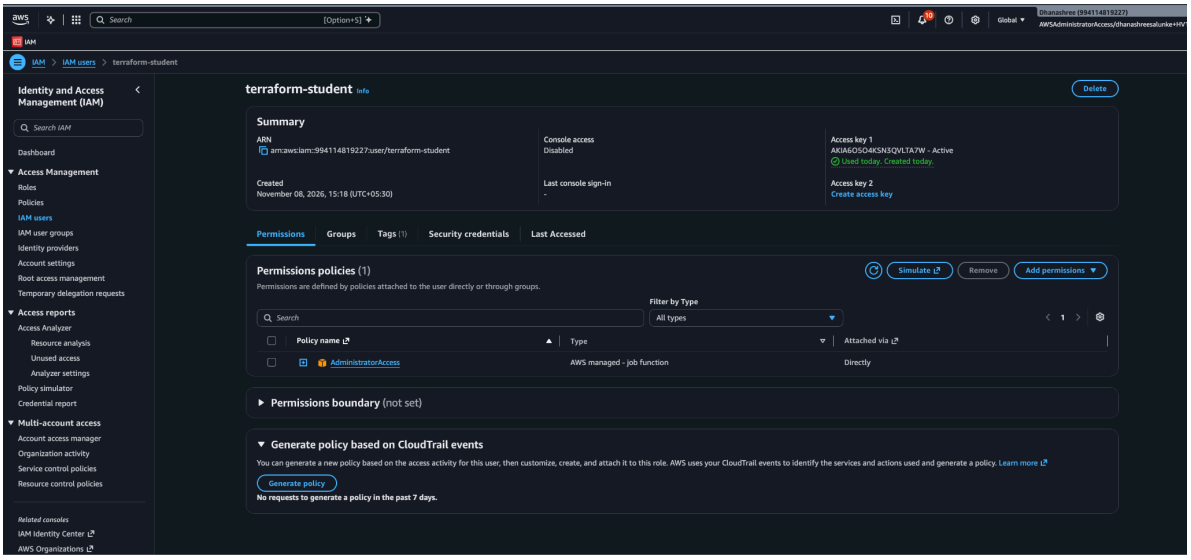


Figure 5: *IAM user used for Terraform*

Terminal output of: aws sts get-caller-identity

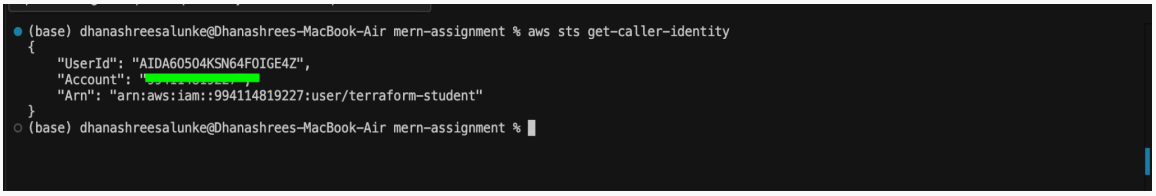


Figure 6: *AWS CLI authenticated successfully*

Terminal output of: terraform init ending with 'Terraform has been successfully initialized!' and terraform validate showing 'Success!'.

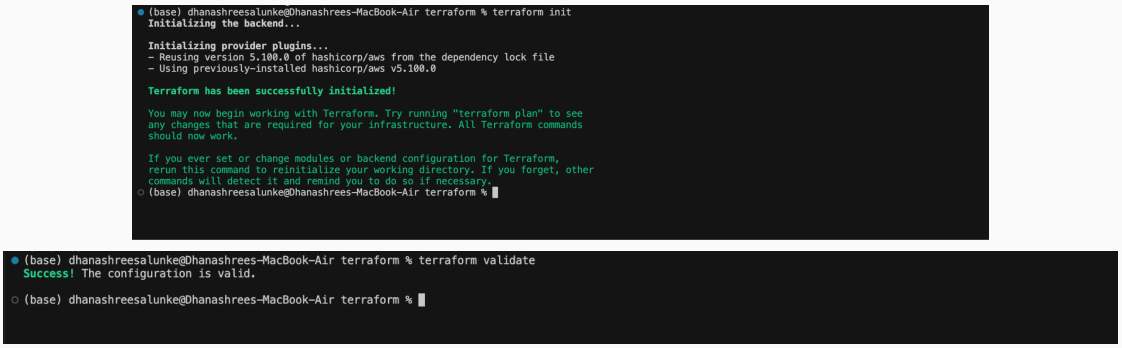


Figure 7: *Terraform initialisation and validation*

EXPLANATION

Files created: `providers.tf` (AWS provider and default tags), `variables.tf` (region, CIDR ranges, my IP, key path, instance types), `main.tf` (all resources), `outputs.tf` (IPs and SSH commands), `terraform.tfvars` (my own values, not pushed to Git).

4.2 Task 2: VPC and network configuration

The VPC `10.0.0.0/16` contains a public subnet (`10.0.1.0/24`) and a private subnet (`10.0.2.0/24`). The public route table sends `0.0.0.0/0` to the Internet Gateway. The private route table sends `0.0.0.0/0` to the NAT Gateway, which lives in the public subnet and uses its own Elastic IP.

VPC > Your VPCs: the VPC named mern-travelmemory-vpc with its CIDR.

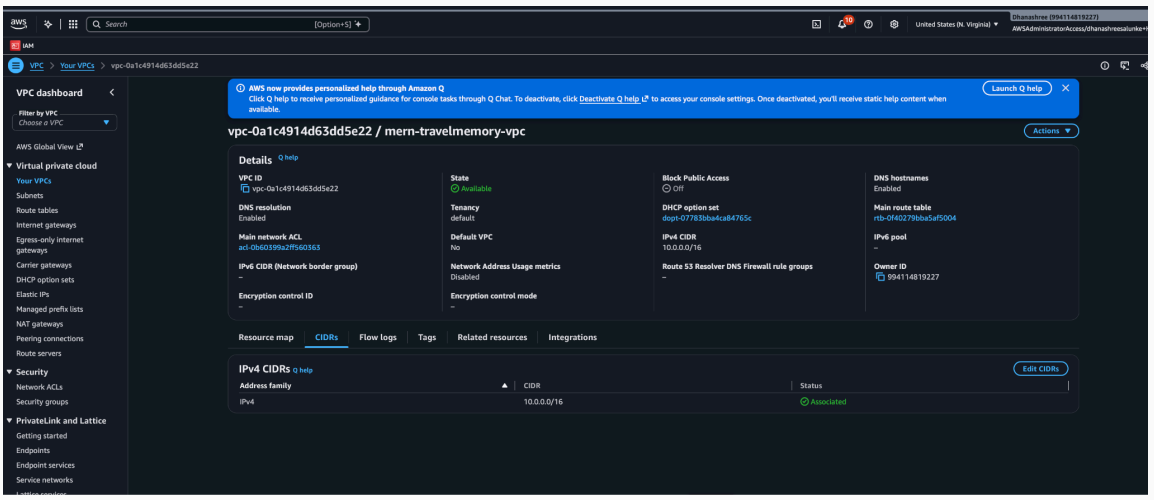


Figure 8: *VPC created by Terraform*

VPC > Subnets: the public and private subnets with their CIDR ranges and availability zone.

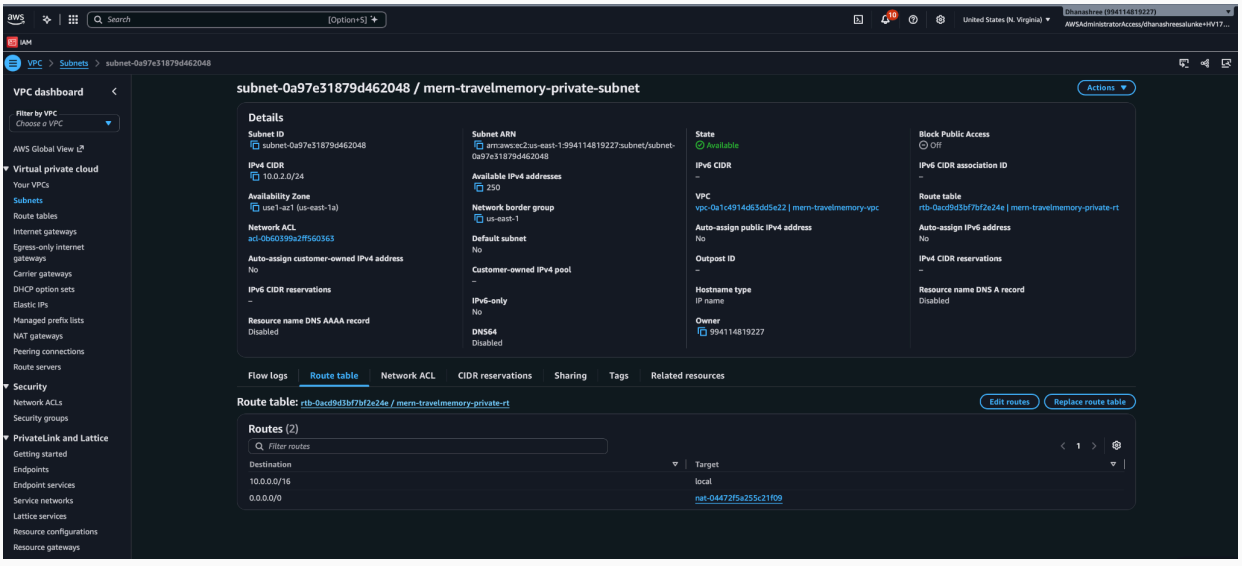


Figure 9: *Public and private subnets*

VPC > Internet gateways and NAT gateways: both attached/available

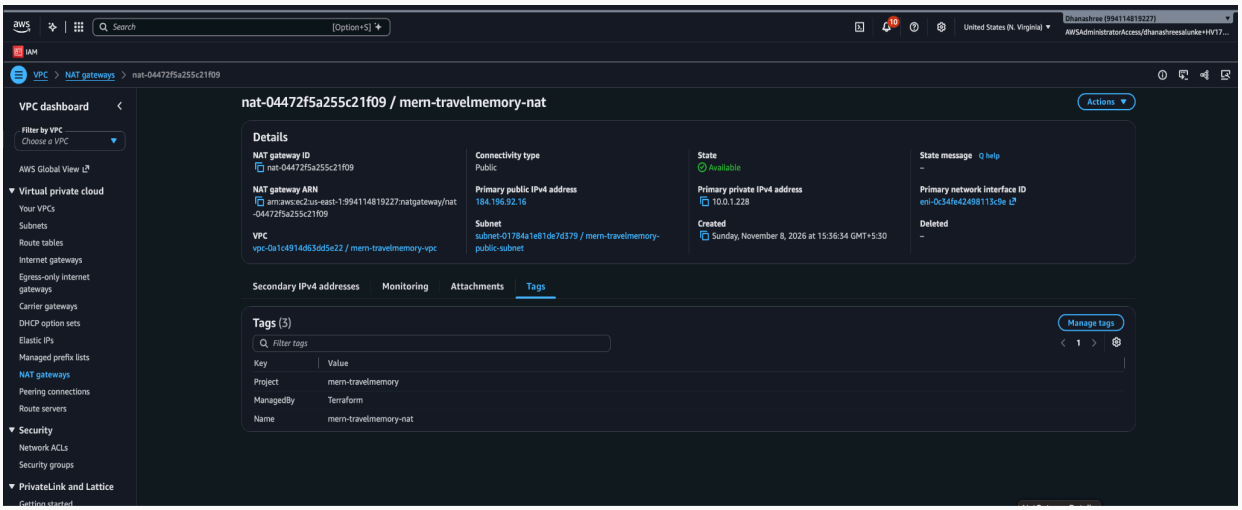


Figure 10: *Internet Gateway and NAT Gateway*

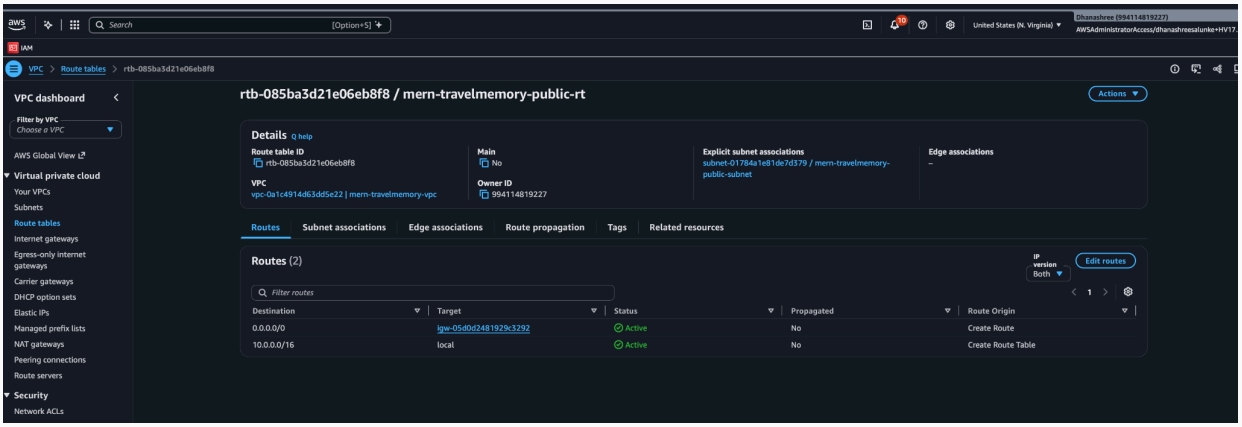


Figure 11: *Public route table: default route to the Internet Gateway*

VPC > Route tables: the Routes tab of the private route table (0.0.0.0/0 to nat-...).

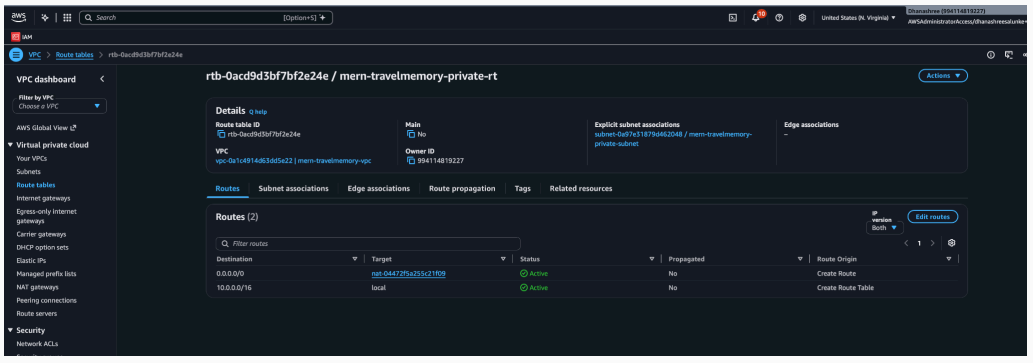


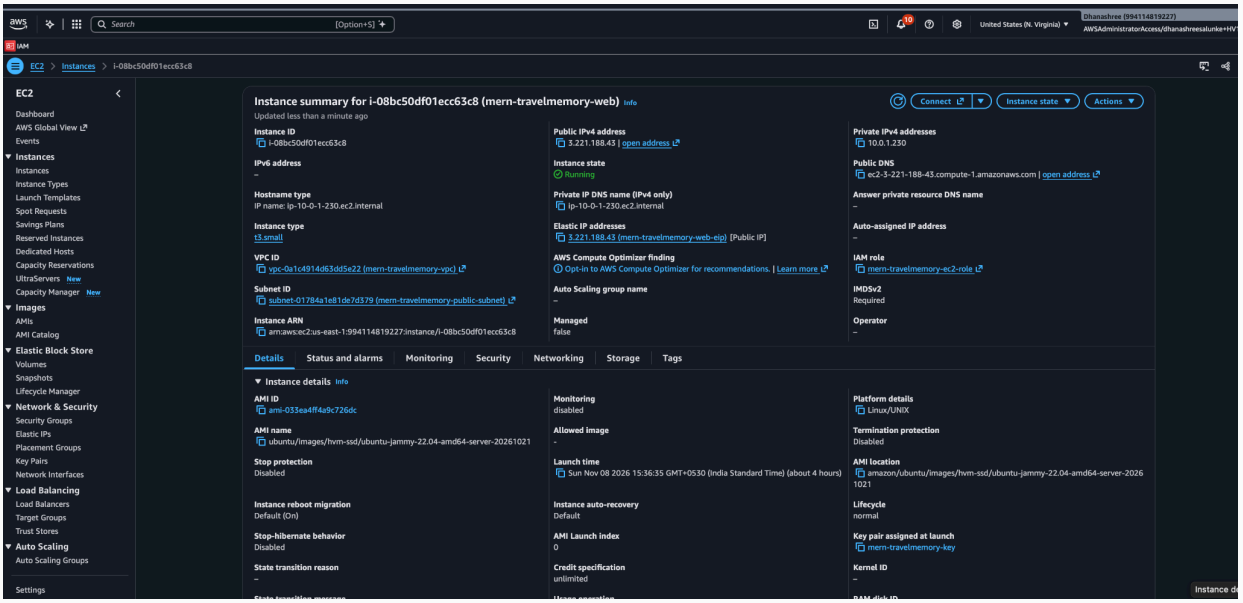
Figure 12: *Private route table: default route to the NAT Gateway*

4.3 Task 3: EC2 instance provisioning

Two Ubuntu 22.04 instances were launched: the **web server** in the public subnet (with an Elastic IP) and the **database server** in the private subnet (no public IP). Both use the same key pair. SSH to the web server is allowed only from my own IP; the database server is reached by jumping through the web server.

EC2 > Instances: both instances Running. Show the Name, Instance type, Public IPv4 and Private IP columns

EC2 Instance 1



EC2 Instance 2

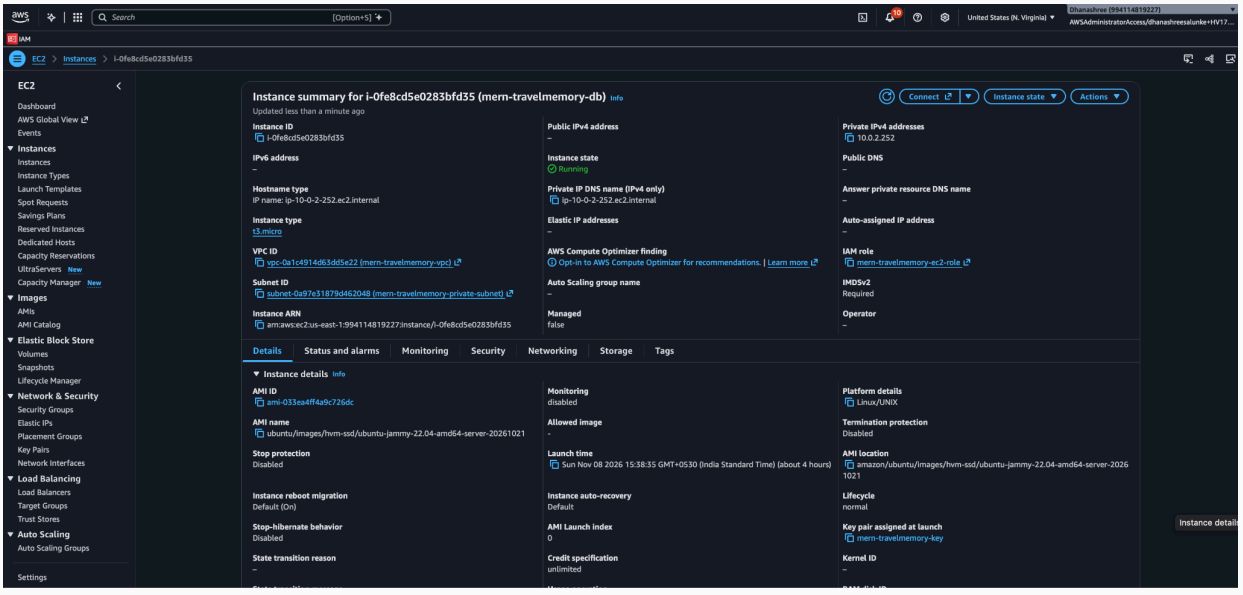


Figure 13: *Web and database EC2 instances*

Terminal: successful ssh login to the web server

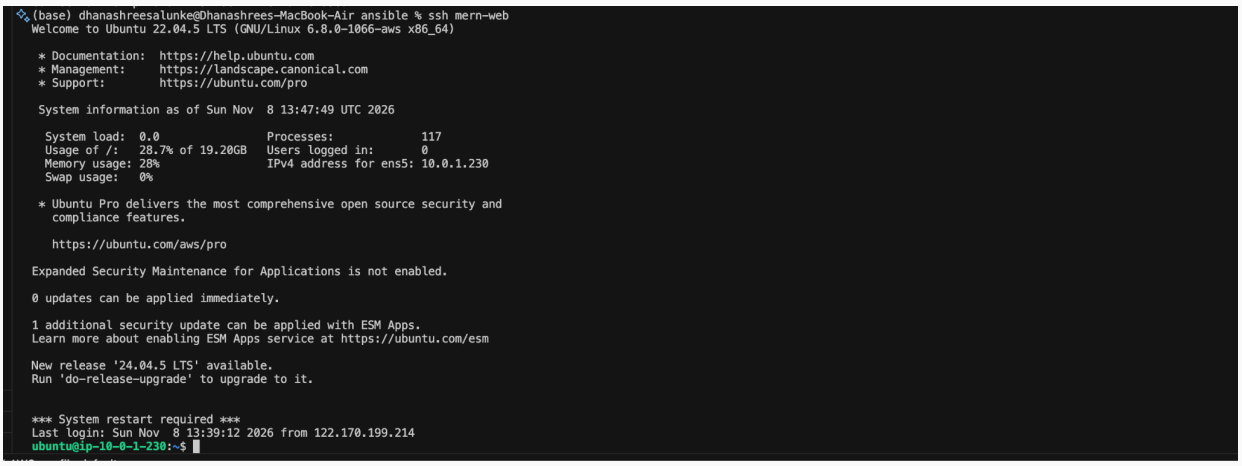


Figure 14: *SSH to the web server*

successful SSH to the database server through the web server (jump host). Show hostname -I returning a 10.0.2.x address.

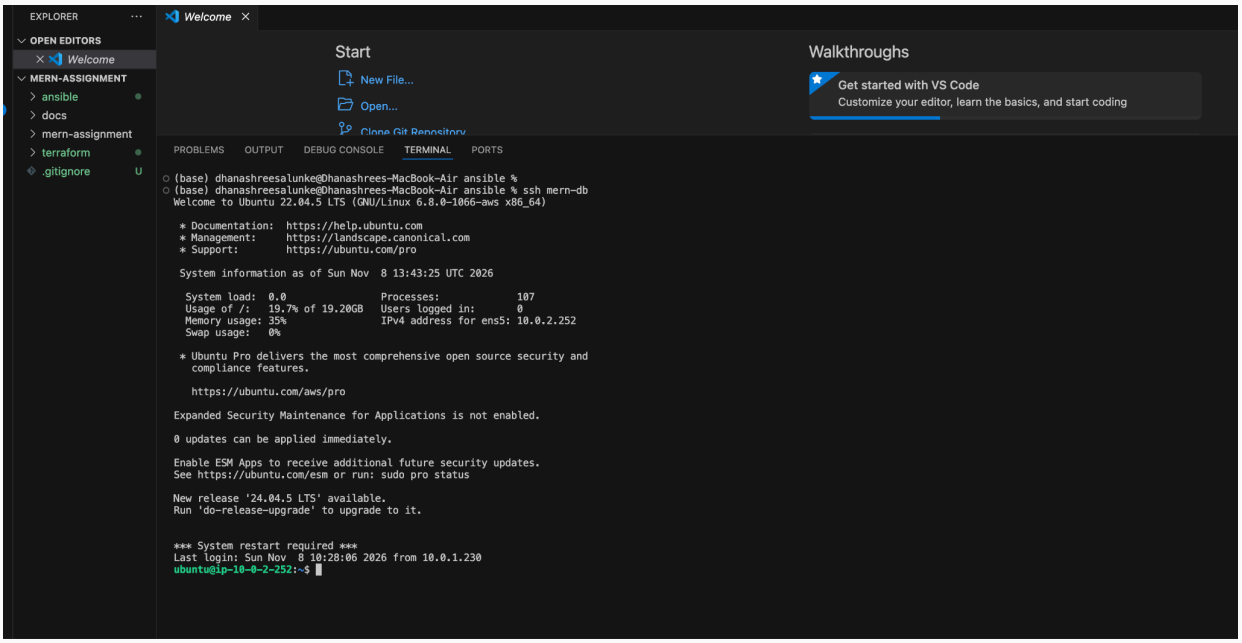


Figure 15: *SSH to the private database server via the jump host*

4.4 Task 4: Security groups and IAM roles

Security group	Inbound rules	Purpose
`mern-travelmemory-web-sg`	22 from my IP only; 80 and 443 from anywhere	Web server firewall
`mern-travelmemory-db-sg`	22 and 27017 only from the web security group	Database firewall

EC2 > Security Groups > web-sg > Inbound rules tab.

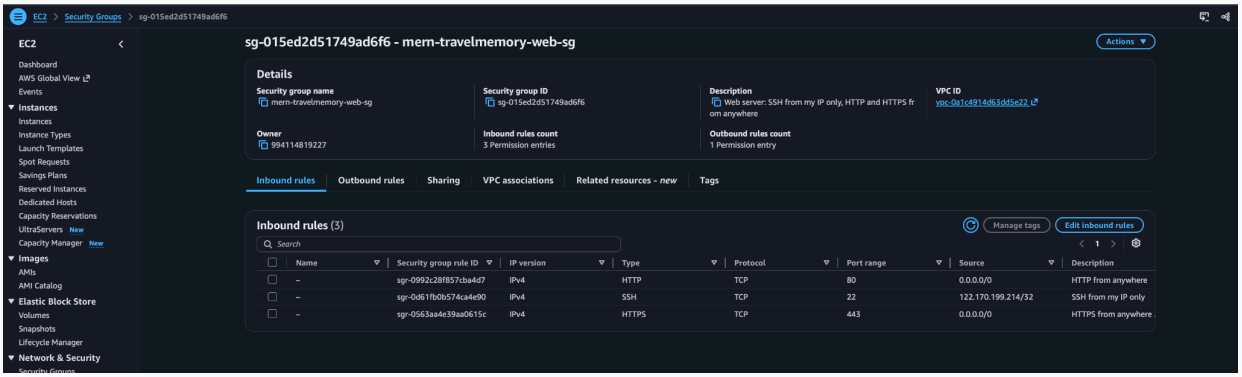


Figure 16: *Web server security group: inbound rules*

EC2 > Security Groups > db-sg > Inbound rules tab

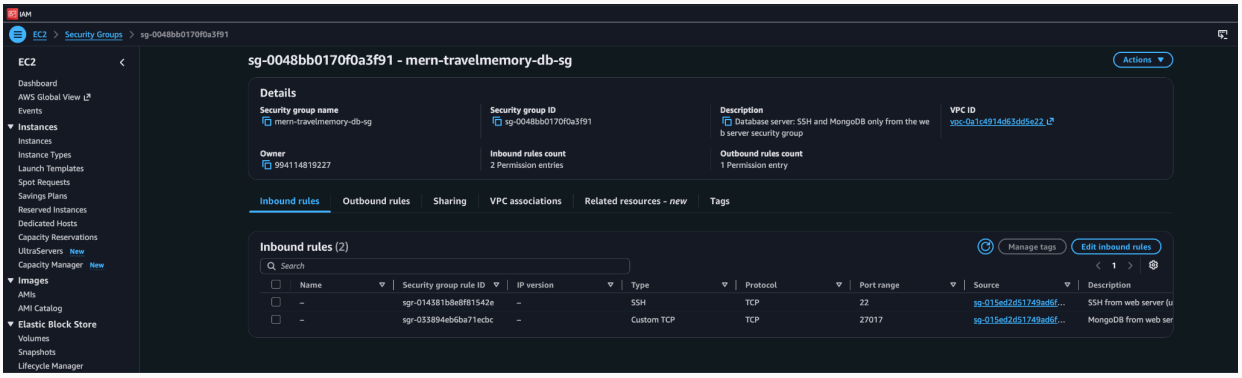


Figure 17: *Database security group: inbound rules*

IAM > Roles > mern-travelmemory-ec2-role > Permissions tab

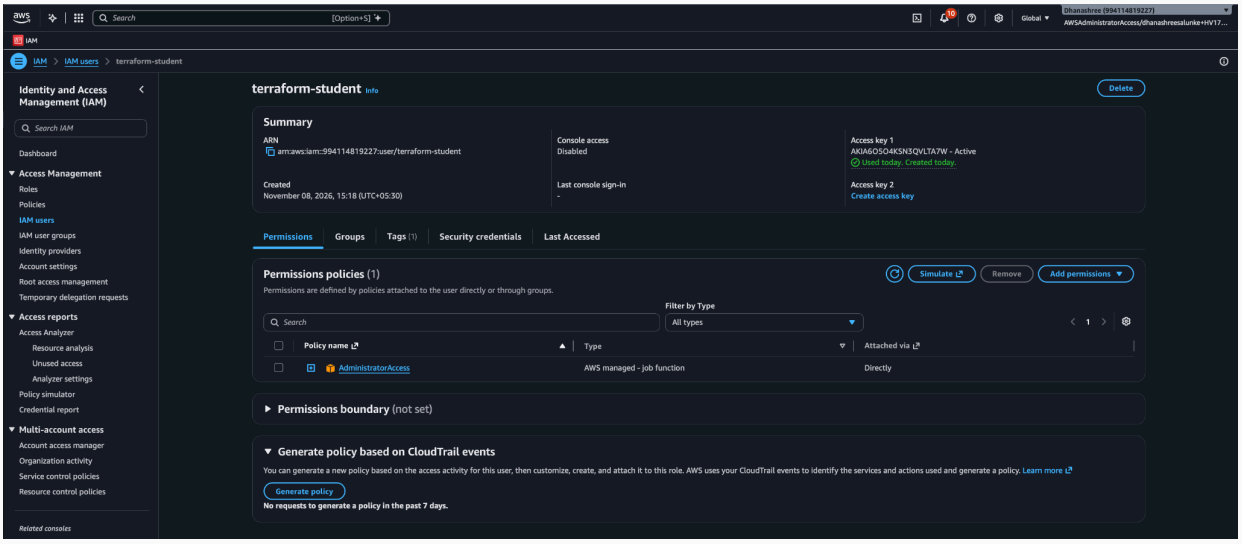


Figure 18: *IAM role attached to the EC2 instances*

EXPLANATION

The IAM role has `AmazonSSMManagedInstanceCore` (console shell access as a backup to SSH) and `CloudWatchAgentServerPolicy`. No AWS access keys are stored on the servers; permissions come from the instance profile.

4.5 Task 5: Resource apply

Terminal output of: terraform apply

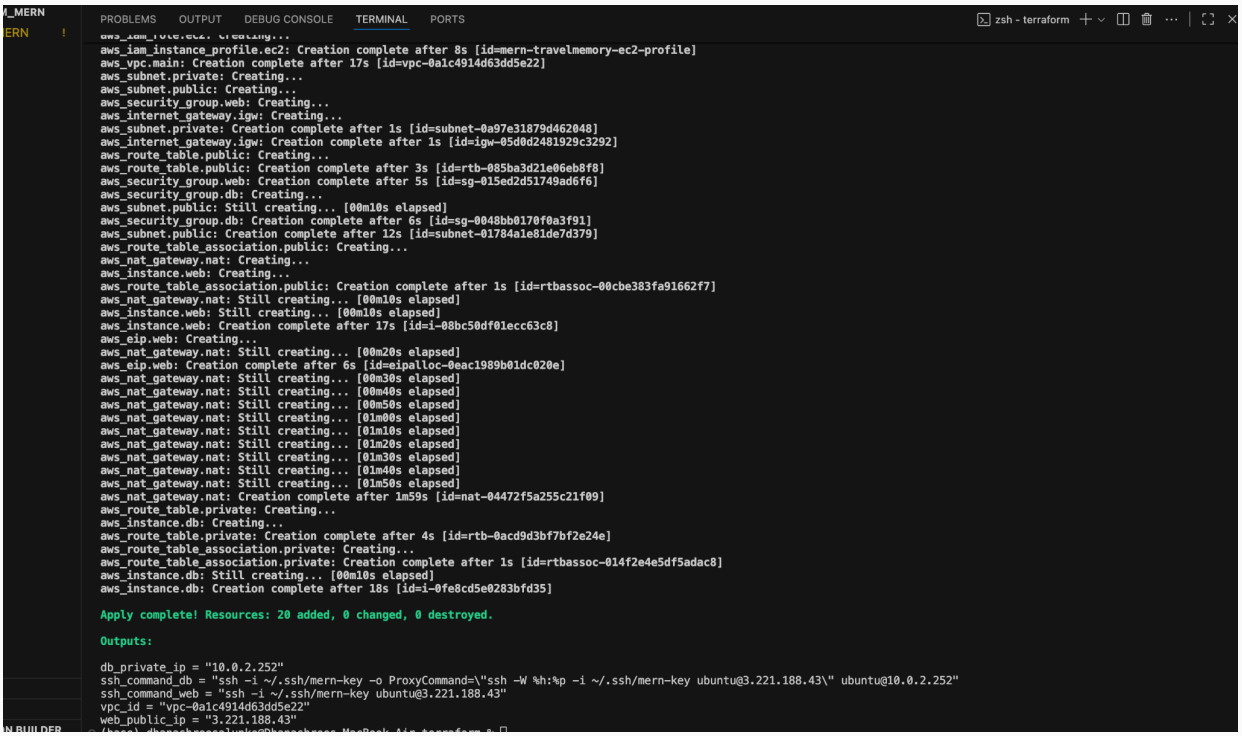


Figure 19: *terraform apply completed*

Terminal output of: terraform output showing web_public_ip, db_private_ip and the SSH commands.

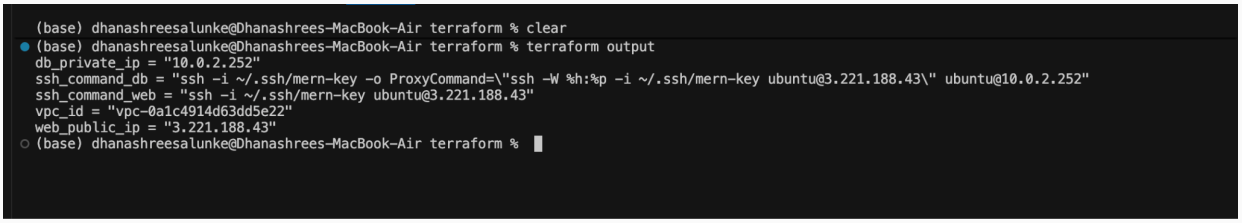


Figure 20: *Terraform output: public IP of the web server*

5. Part 2: Configuration and deployment with Ansible

5.1 Task 1: Ansible configuration

Ansible connects over SSH using the same key. Because the database server has no public IP, its inventory entry uses a `ProxyCommand` that hops through the web server. The database passwords are stored in an encrypted **Ansible Vault** file, and shared settings are in `inventory/group_vars/all/vars.yml`.

VS Code showing ansible.cfg and inventory/hosts.in

```
(base) dhanashreesalunke@Dhanashrees-MacBook-Air ansible % cat ansible.cfg
[defaults]
inventory            = inventory/hosts.ini
remote_user          = ubuntu
private_key_file     = ~/.ssh/mern-key
interpreter_python   = auto_silent
retry_files_enabled  = False
# Uncomment after you create a .vault_pass file (it is git-ignored) to skip typing the vault password:
vault_password_file = .vault_pass

[privilege_escalation]
become        = True
become_method = sudo
become_user   = root

[ssh_connection]
pipelining = True
ssh_args   = -o ControlMaster=auto -o ControlPersist=60s -o StrictHostKeyChecking=accept-new

(base) dhanashreesalunke@Dhanashrees-MacBook-Air ansible %
```

```
(base) dhanashreesalunke@Dhanashrees-MacBook-Air ansible % cat inventory/group_vars/all/vars.yml
---
# ---- Application ----
app_repo: "https://github.com/UnpredictablePrashant/TravelMemory.git"
app_branch: main
app_dir: /opt/travelmemory
backend_port: 3001
node_major: 22

# ---- Network (must match your Terraform values) ----
public_subnet_cidr: "10.0.1.0/24"
web_public_ip: "{{ hostvars['web1'].ansible_host }}"
db_private_ip: "{{ hostvars['db1'].ansible_host }}"

# ---- MongoDB ----
mongo_version: "7.0"
mongo_db_name: travelmemory
mongo_admin_user: mongoadmin
mongo_app_user: travelapp
# Passwords live in vault.yml (encrypted with ansible-vault):
#   vault_mongo_admin_password
#   vault_mongo_app_password

(base) dhanashreesalunke@Dhanashrees-MacBook-Air ansible %
```

Figure 21: *Ansible configuration and inventory*

Terminal: head -1 inventory/group_vars/all/vault.yml showing $ANSIBLE_VAULT;1.1;AES256 (proves the secrets file is encrypted).

```
(base) dhanashreesalunke@Dhanashrees-MacBook-Air ansible % head -1 inventory/group_vars/all/vault.yml
$ANSIBLE_VAULT;1.1;AES256
(base) dhanashreesalunke@Dhanashrees-MacBook-Air ansible %
```

Figure 22: *Encrypted Ansible Vault file*

Terminal output of: ansible all -m ping with web1 | SUCCESS and db1 | SUCCESS (pong).

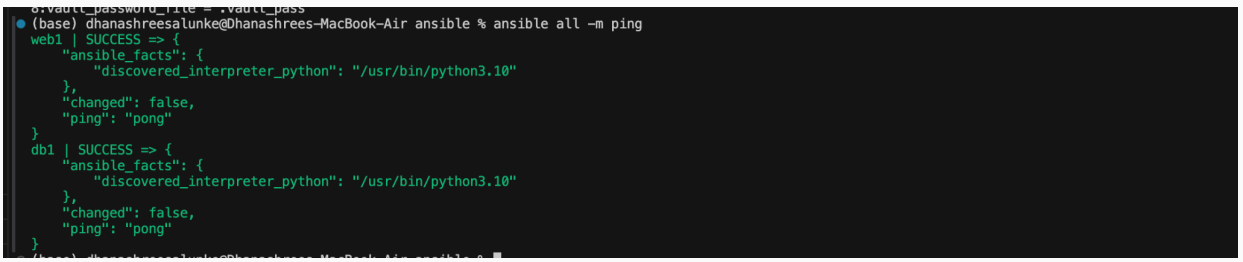


Figure 23: *Ansible can reach both servers*

5.2 Task 2: Web server setup (Node.js, NPM, clone, dependencies)

Playbook `01-common.yml` installs base packages on both servers. Playbook `03-webserver.yml` adds a swap file, installs Node.js (NodeSource repository) with NPM, Nginx and PM2, clones the TravelMemory repository into `/opt/travelmemory`, and runs `npm install` for the backend and the frontend.

Terminal: end of ansible-playbook playbooks/01-common.yml with the PLAY RECAP (failed=0, unreachable=0).

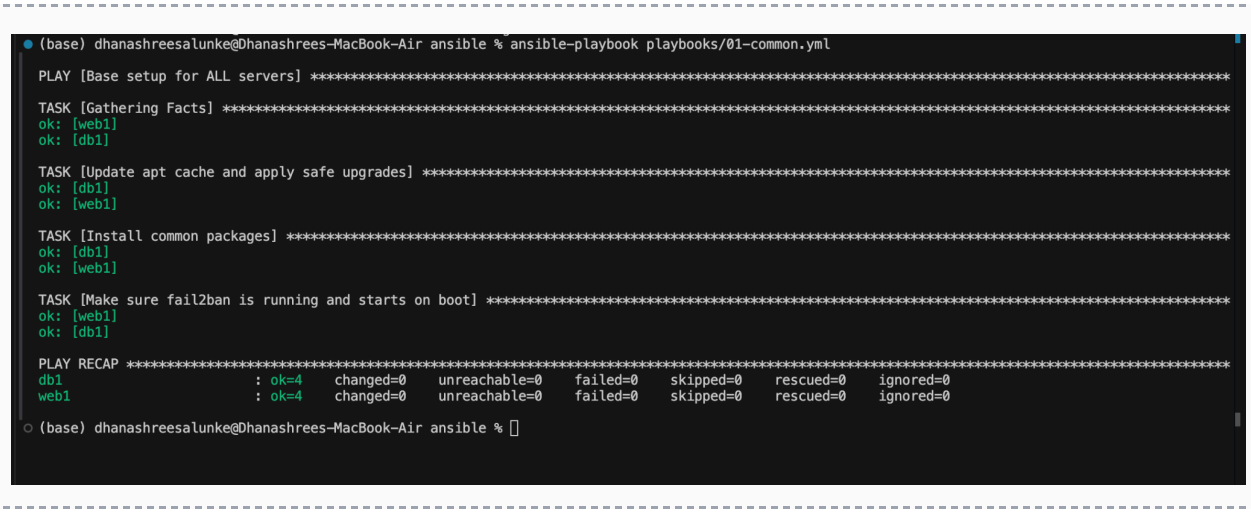


Figure 24: *Playbook 01: common setup recap*

Terminal on the web server: node -v, npm -v, nginx -v, pm2 -v.

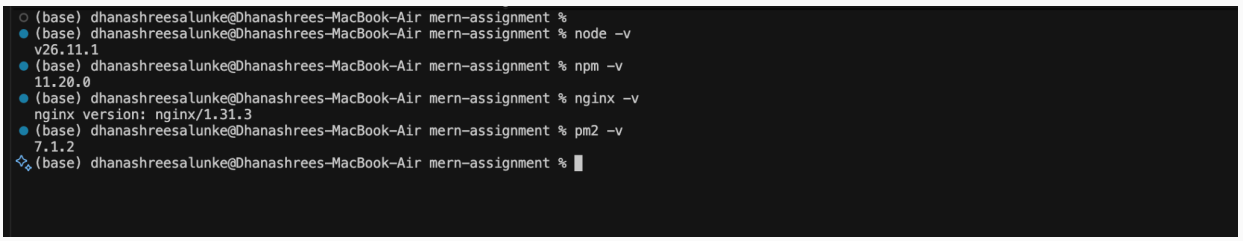


Figure 25: *Node.js, NPM, Nginx and PM2 installed*

Terminal on the web server: ls /opt/travelmemory showing the backend and frontend folders.

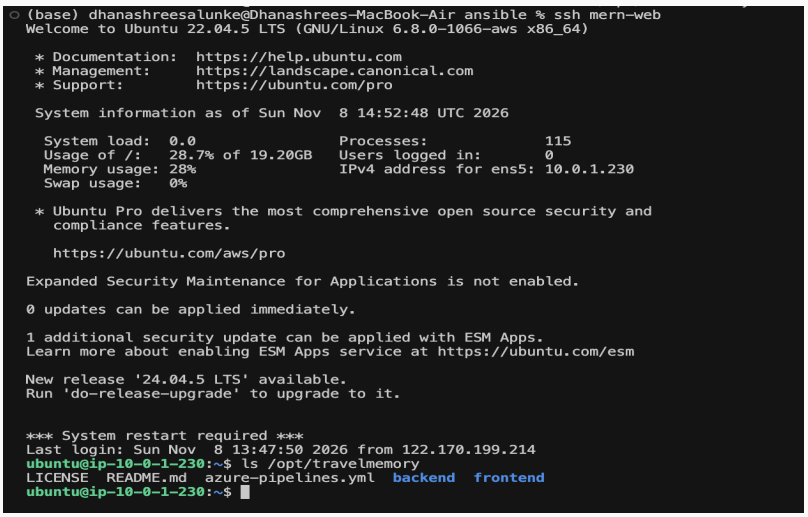


Figure 26: *TravelMemory repository cloned*

5.3 Task 3: Database server setup (MongoDB, security, users)

Playbook `02-database.yml` adds the official MongoDB 7.0 repository, installs MongoDB, creates an **admin user** and an **application user** (`travelapp`, `readWrite` on the `travelmemory` database only), then enables authentication and binds MongoDB to its private IP.

Terminal: end of ansible-playbook playbooks/02-database.yml with PLAY RECAP and the 'MongoDB login test returned: 1' message.



Figure 27: *Playbook 02: MongoDB install and securing*

On the DB server: sudo systemctl status mongod --no-pager (active/running) and sudo ss -tlnp | grep 27017



Figure 28: *MongoDB service running and bound to private IP*

On the DB server: mongosh --quiet --eval "db.adminCommand('listDatabases')" failing with an authentication error.

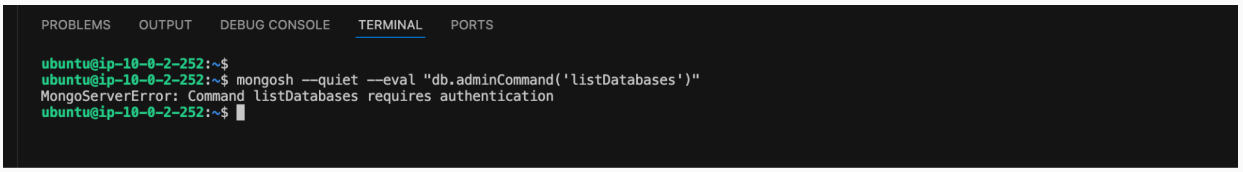


Figure 29: *Anonymous access to MongoDB is refused*

On the DB server: mongosh -u travelapp --authenticationDatabase travelmemory travelmemory, then show collections

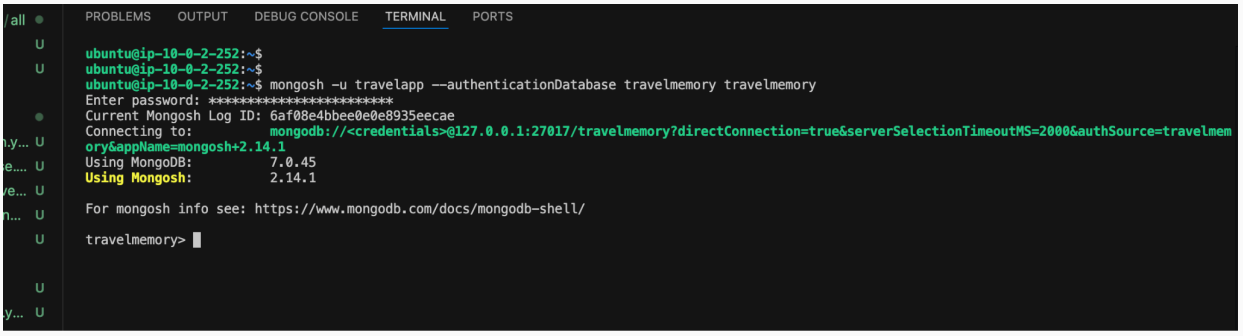


Figure 30: *Application user can log in and sees the trips collection*

5.4 Task 4: Application deployment

Playbook `03-webserver.yml` writes `backend/.env` (`MONGO_URI`, `PORT=3001`, `CORS_ORIGIN`) and `frontend/.env` (`REACT_APP_BACKEND_URL=http://<web-ip>/api`), builds the React app, starts the backend with PM2 (and enables start on reboot), and configures Nginx to serve the React build and proxy `/api/` to the backend.

Terminal on the web server: sed 's#//.@#//***@#' /opt/travelmemory/backend/.env and cat /opt/travelmemory/frontend/.env (password must be hidden).*

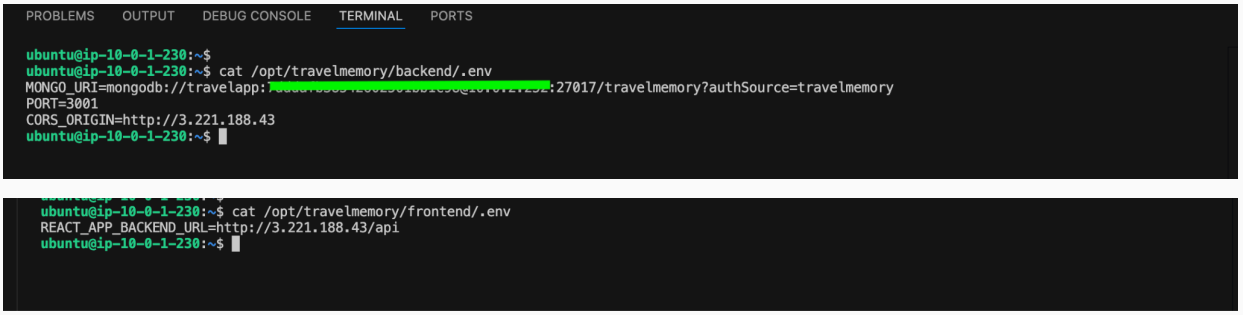


Figure 31: *Environment variables configured*

Terminal on the web server: pm2 status showing travel-backend online.

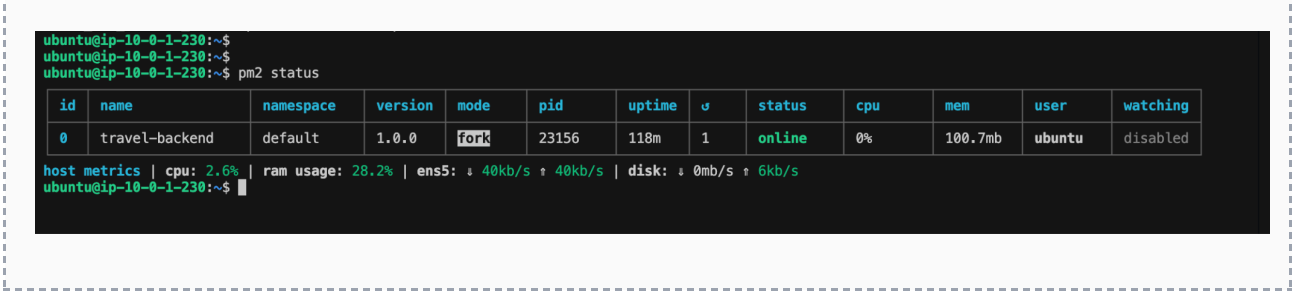


Figure 32: *Backend running under PM2*

Terminal on the web server: sudo nginx -t and the contents of /etc/nginx/sites-available/travelmemory

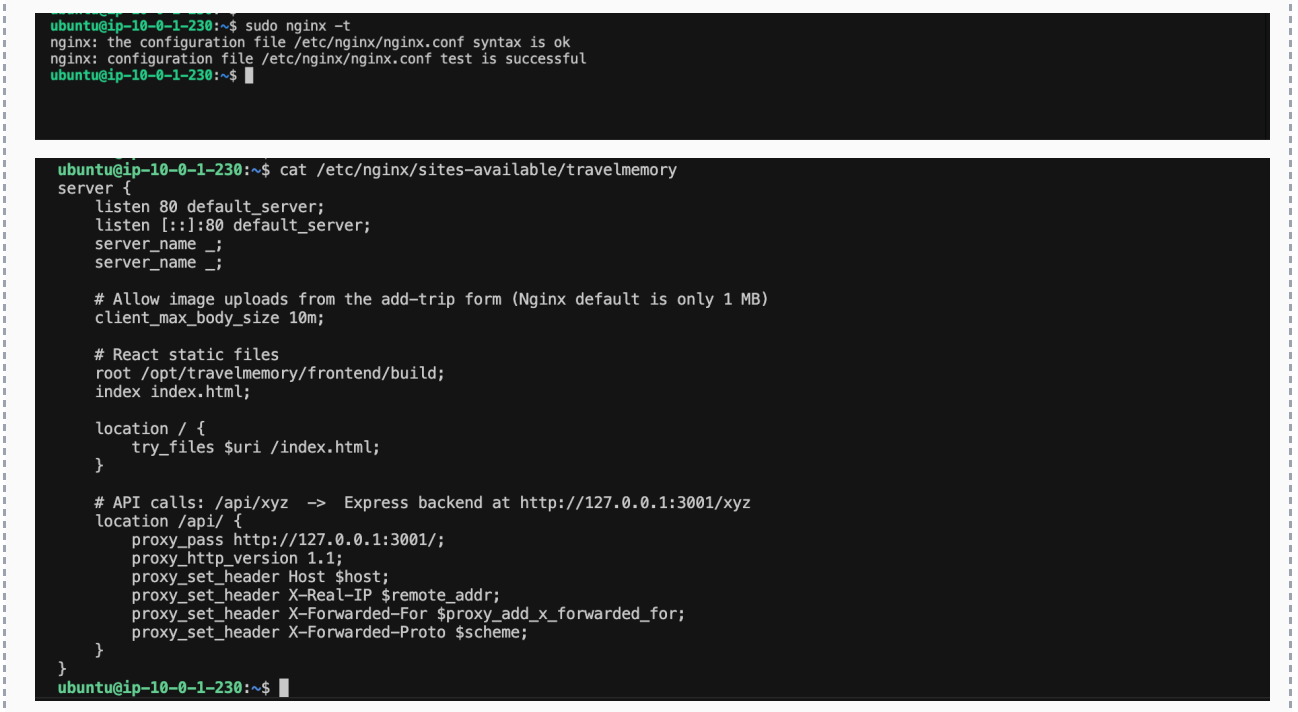


Figure 33: *Nginx reverse proxy configuration*

Terminal: end of ansible-playbook playbooks/03-webserver.yml with PLAY RECAP (failed=0).



Figure 34: *Playbook 03: web server and application recap*

EXPLANATION

The frontend calls `/api/...` on the same address as the page. Nginx removes `/api` and forwards the request to Express on port 3001, so port 3001 never has to be opened to the internet. `CORS_ORIGIN` in the backend `.env` tells Express which website address is allowed to call it.

5.5 Task 5: Security hardening

Playbook `04-hardening.yml` disables root login and password authentication for SSH, restricts SSH to the `ubuntu` user, enables the UFW host firewall (deny incoming by default), enables automatic security updates and keeps fail2ban running. Security groups (Terraform) and UFW (Ansible) work as two layers.

Terminal: end of ansible-playbook playbooks/04-hardening.yml with PLAY RECAP.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(base) dhanashreesalunke@Dhanashrees-MacBook-Air ansible %
(base) dhanashreesalunke@Dhanashrees-MacBook-Air ansible % ansible-playbook playbooks/04-hardening.yml

PLAY [Security hardening for ALL servers] ***************************************************************************************************************

TASK [Gathering Facts] **********************************************************************************************************************************
ok: [web1]
ok: [db1]

TASK [Harden the SSH daemon (key login only, no root login)] ********************************************************************************************
ok: [db1]
ok: [web1]

TASK [UFW - rate-limit SSH on the web server] ************************************************************************************************************
skipping: [db1]
ok: [web1]

TASK [UFW - allow HTTP and HTTPS on the web server] ******************************************************************************************************
skipping: [db1] => (item=80)
skipping: [db1] => (item=443)
skipping: [db1]
ok: [web1] => (item=80)
ok: [web1] => (item=443)

TASK [UFW - on the DB server allow SSH and MongoDB only from the public subnet] **************************************************************************
skipping: [web1] => (item=22)
skipping: [web1] => (item=27017)
skipping: [web1]
ok: [db1] => (item=22)
ok: [db1] => (item=27017)

TASK [UFW - deny everything else coming in, then switch the firewall on] *********************************************************************************
ok: [web1]
ok: [db1]

TASK [Enable unattended security upgrades] ***************************************************************************************************************
ok: [web1]
ok: [db1]

TASK [Make sure fail2ban is running] *********************************************************************************************************************
ok: [db1]
ok: [web1]

PLAY RECAP ***********************************************************************************************************************************************
db1                        : ok=6    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
web1                       : ok=7    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

(base) dhanashreesalunke@Dhanashrees-MacBook-Air ansible %
(base) dhanashreesalunke@Dhanashrees-MacBook-Air ansible %
```

Figure 35: *Playbook 04: hardening recap*

On the web server: sudo ufw status verbose.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

ubuntu@ip-10-0-1-230:~$
ubuntu@ip-10-0-1-230:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     LIMIT IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
22/tcp (v6)                LIMIT IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)

ubuntu@ip-10-0-1-230:~$
```

Figure 36: *UFW firewall status: web server*

On the DB server: sudo ufw status verbose (22 and 27017 allowed only from 10.0.1.0/24).

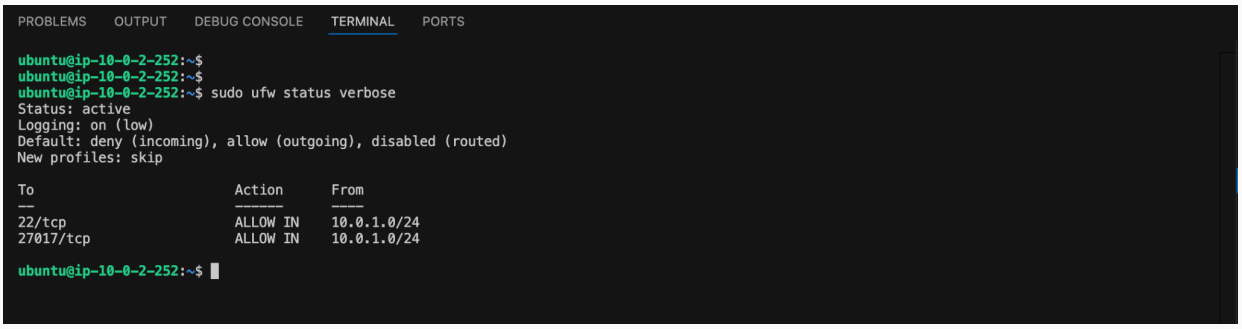


Figure 37: *UFW firewall status: database server*

From local PC: nc -zv <web-ip> 27017 and nc -zv <web-ip> 3001 failing, nc -zv <web-ip> 80 succeeding; ssh root@<web-ip> denied.

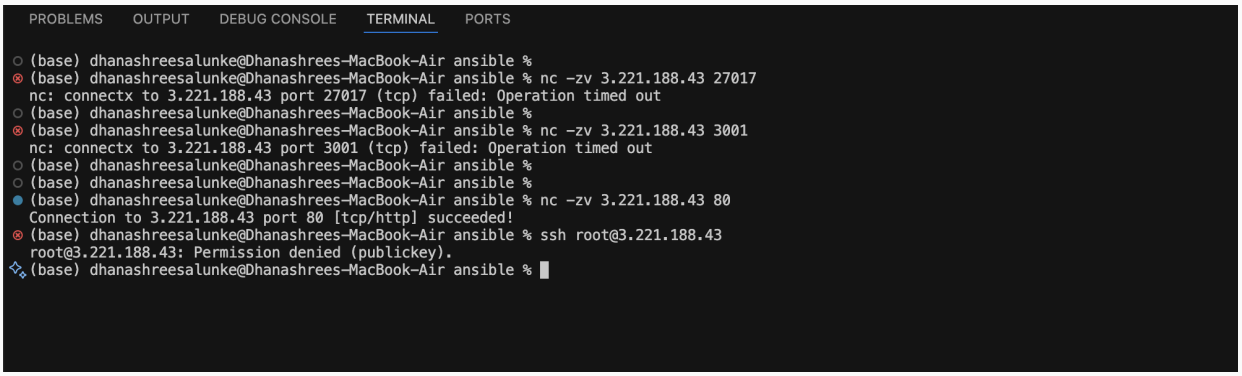


Figure 38: *Negative tests: closed ports and root login denied*

6. How the application components interact

This table describes each hop of a request, the port used and the rule that allows it.

From	To	Port / protocol	Allowed by
User's browser	Nginx on the web server	80 / HTTP	Web security group (0.0.0.0/0), UFW allow 80
Nginx	React build files	local disk	Files in `/opt/travelmemory/frontend/build`
Browser (React code)	Nginx `/api/` path	80 / HTTP	Same as above, no CORS preflight issues
Nginx	Express backend	3001 / HTTP (localhost)	Never exposed outside the server
Express backend	MongoDB on the DB server	27017 / MongoDB	DB security group (source: web security group), UFW allow from 10.0.1.0/24
DB server	Internet (package updates)	443, 80 outbound	Private route table to NAT Gateway; no inbound path
Admin (me)	Web server	22 / SSH	Web security group (my IP only), key-based login
Admin (me)	DB server	22 / SSH via jump host	DB security group (web SG only)

7. Working application: proof of deployment

The application was opened using the web server's public IP address. A new trip was added through the UI and was still present after refreshing the page, which proves that React, Express and MongoDB are working together.

Browser at http://<web-public-ip>/ showing the TravelMemory home page. Include the address bar.

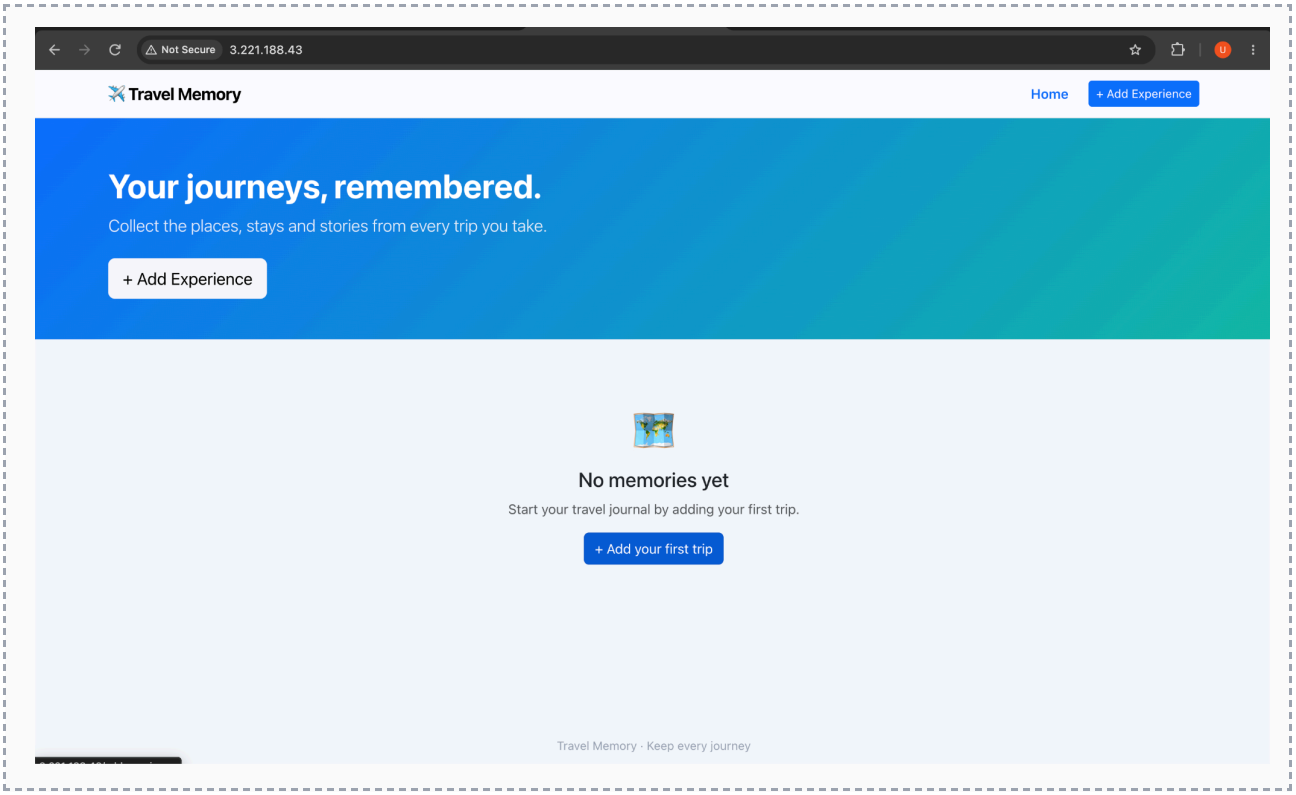


Figure 39: *TravelMemory home page served from the EC2 web server*

The 'Add experience' form filled in with a sample trip

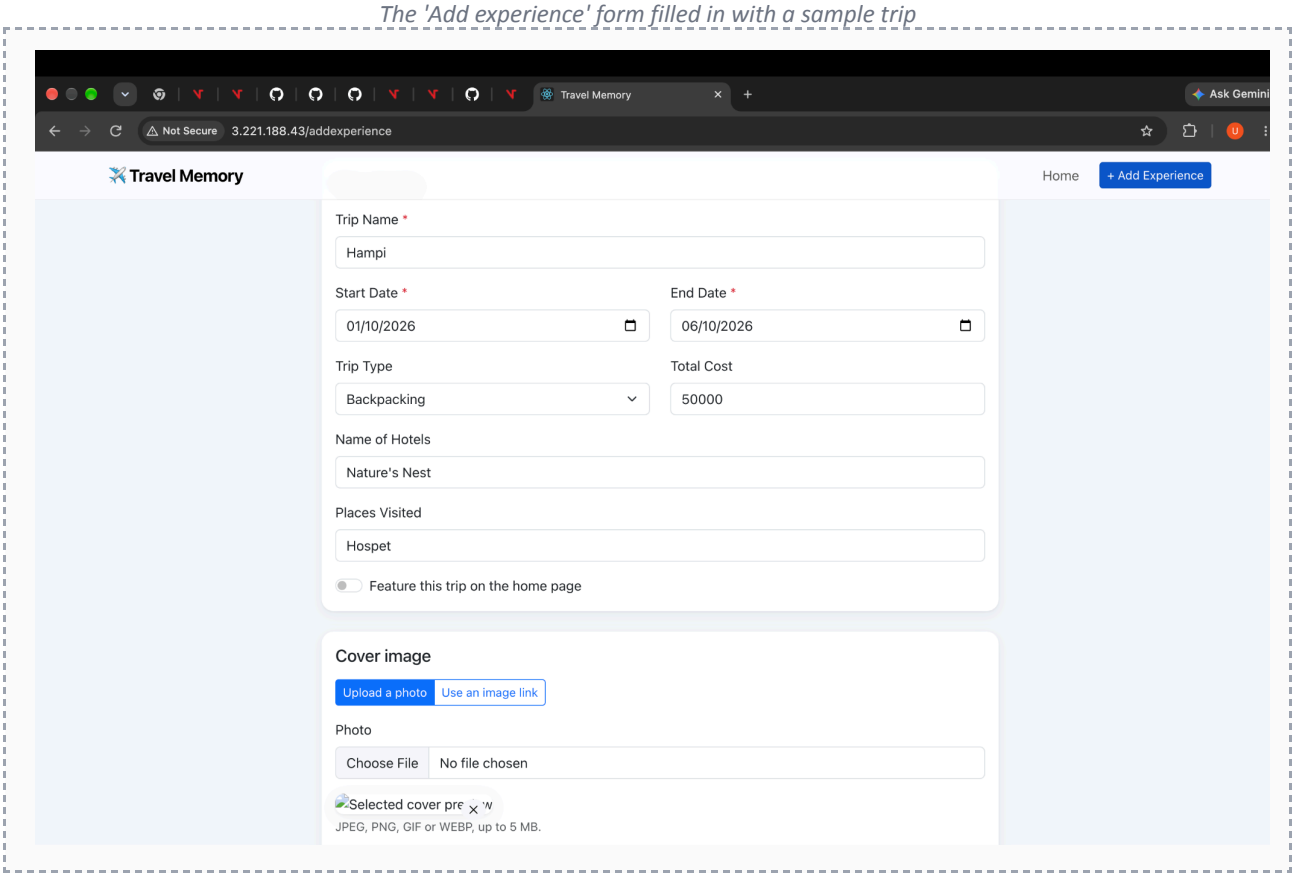


Figure 40: *Adding a new trip through the UI*

Home page after saving, showing the new trip card.

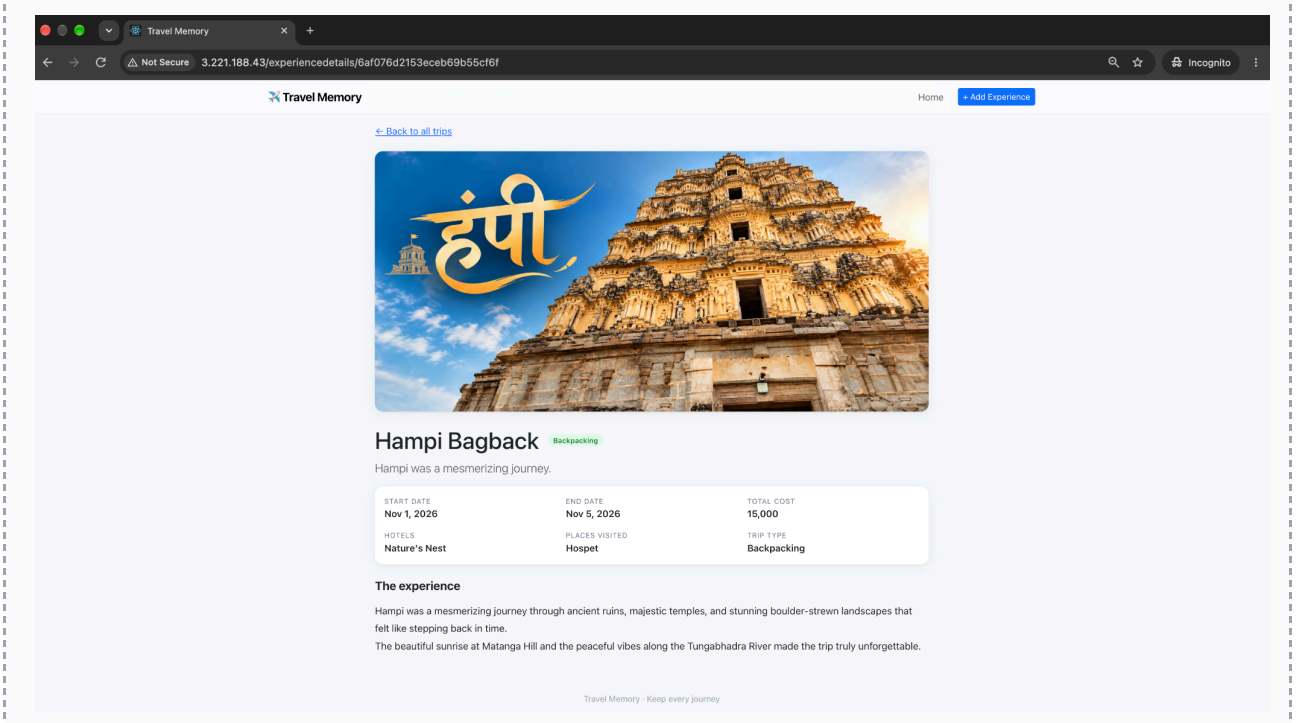


Figure 41: *New trip saved and displayed*

Browser after a refresh (Cmd+Shift+R) or the trip details page, showing the data persisted.

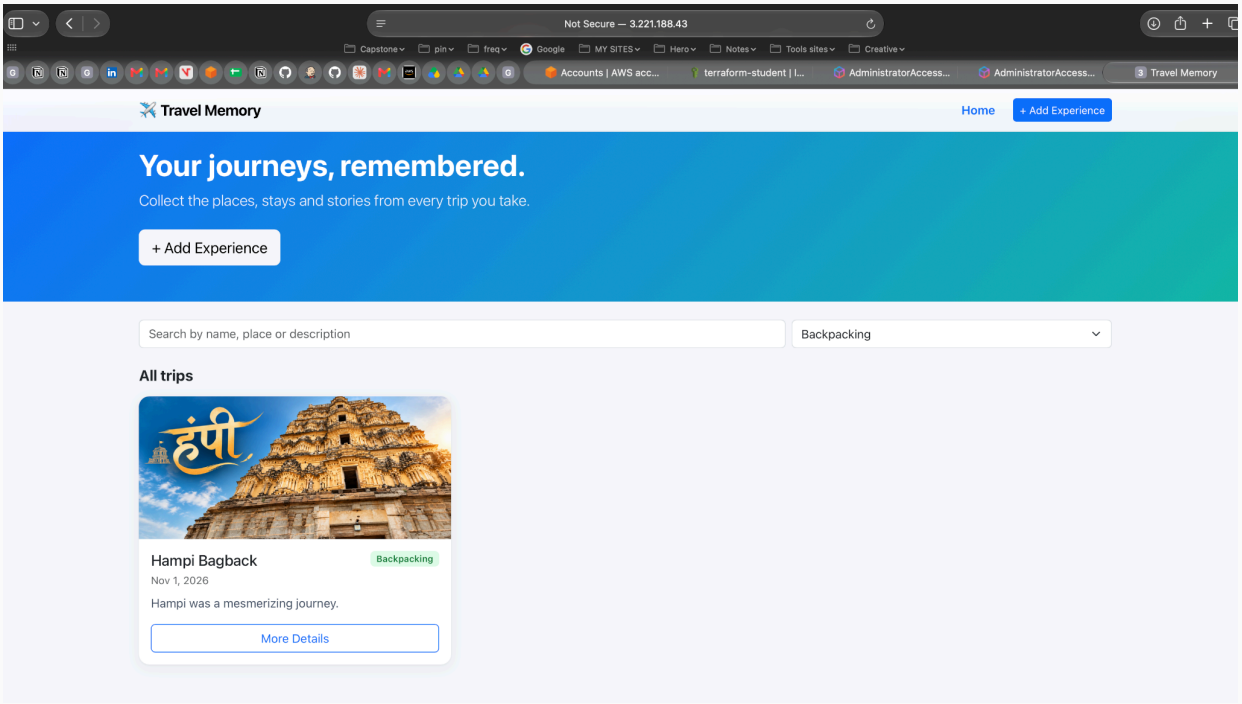


Figure 42: *Data persists after refresh (stored in MongoDB)*

Your architecture diagram showing the request path, or the browser's Network tab for a request to /api/trip (status 200).

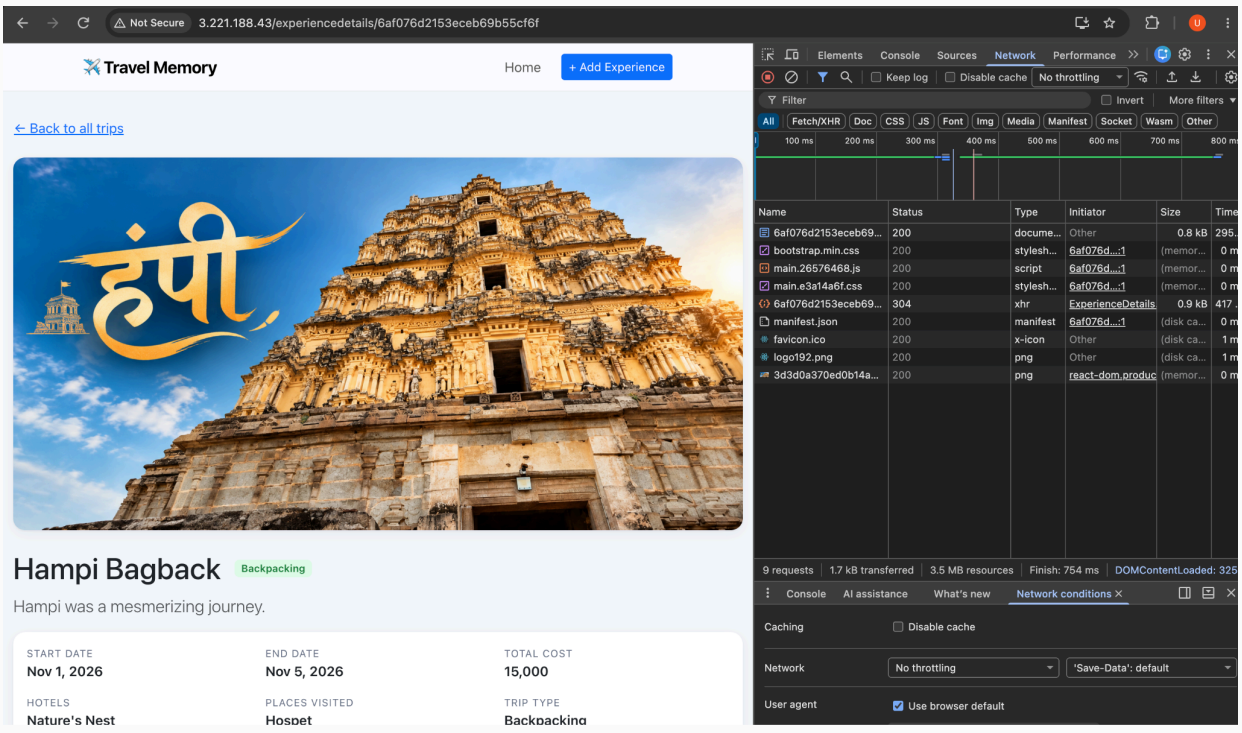


Figure 43: *End-to-end request path*

Browser DevTools (F12) > Network tab showing the POST /api/trip request with status 200/201 and the GET /api/trip request.

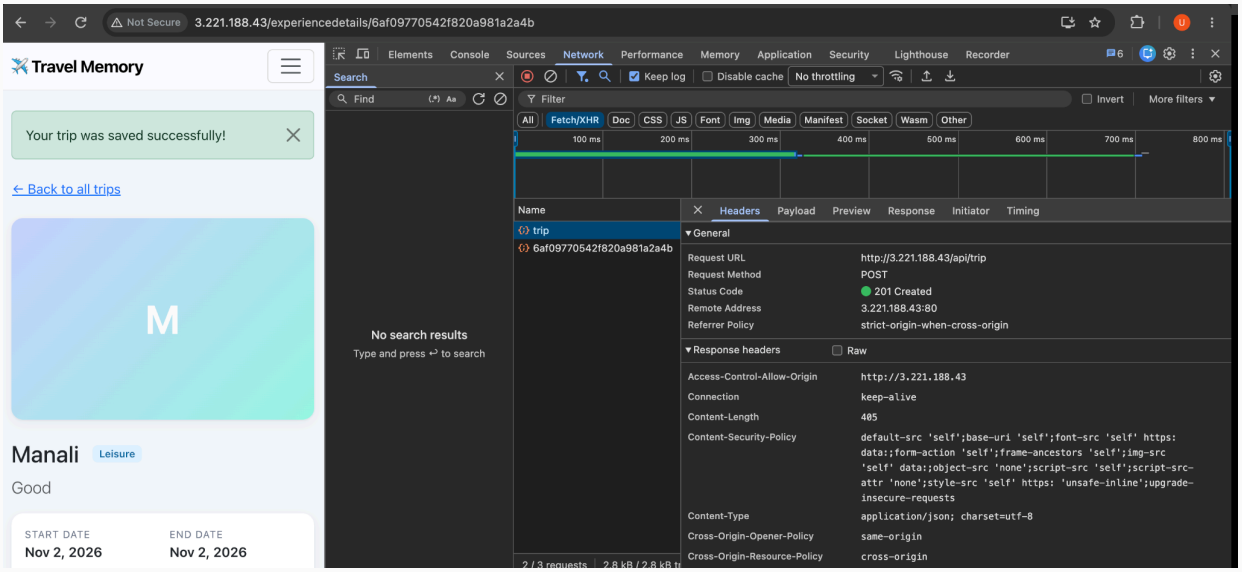


Figure 44: *Frontend communicating with the backend AP*

DB server: mongosh login as travelapp, then db.trips.find().limit(1) showing the stored document.

```
ubuntu@ip-10-0-2-252:~$ mongosh -u travelapp --authenticationDatabase travelmemory travelmemory
Enter password: **************************
Current Mongosh Log ID: 6af0986e344cb76c16d8eaf0
Connecting to:          mongodb://<credentials>@127.0.0.1:27017/travelmemory?directConnection=true&serverSelectionTimeoutMS=2000&authSource=travelmemory&appName=mongosh+2.14.1
Using MongoDB:          7.0.45
Using Mongosh:          2.14.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

travelmemory> show collections
| db.trips.find().limit(1)
tripdetails
travelmemory>
```

```
travelmemory>

travelmemory> db.tripdetails.countDocuments()
| db.tripdetails.find({}, { tripName: 1, createdAt: 1 }).sort({ createdAt: -1 })
[
  {
    _id: ObjectId('6af09770542f820a981a2a4b'),
    tripName: 'Manali',
    createdAt: ISODate('2026-11-08T15:38:24.466Z')
  },
  {
    _id: ObjectId('6af076d2153eceb69b55cf6f'),
    tripName: 'Hampi Bagback',
    createdAt: ISODate('2026-11-08T13:19:14.248Z')
  },
  {
    _id: ObjectId('6af07485153eceb69b55cf69'),
    tripName: 'Test Trip',
    createdAt: ISODate('2026-11-08T13:09:25.677Z')
  }
]
```

Figure 45: *The saved trip stored in MongoDB*

8. Security summary

8.1 Open ports

Port	Service	Web server	DB server
22	SSH	My IP only (SG); rate-limited by UFW	Web security group only (SG); 10.0.1.0/24 (UFW)
80	HTTP (Nginx)	Open to the internet	Closed
443	HTTPS	Open in SG and UFW (no certificate installed)	Closed
3001	Express backend	Not open (reached only through Nginx)	Closed
27017	MongoDB	Closed	Web security group only (SG); 10.0.1.0/24 (UFW)

8.2 Login methods

What	How access works	Protection
Web server (SSH)	SSH key pair, user `ubuntu`	No password login, root login disabled, `AllowUsers ubuntu`, fail2ban, my IP only
DB server (SSH)	SSH key through the web server (jump host)	No public IP; security group allows only the web server
MongoDB	`travelapp` (readWrite on one database); `mongoadmin` for administration	Authentication enabled; passwords in Ansible Vault
AWS console shell (backup)	Session Manager	IAM role; no open port needed

8.3 Additional measures

- Layered firewalls: AWS security groups plus UFW on each server.
- IMDSv2 required and encrypted EBS volumes on both instances.
- Automatic security updates and fail2ban against repeated SSH attempts.
- Secrets are not committed to GitHub: `.gitignore` excludes state files, `terraform.tfvars`, `hosts.ini`, `.vault_pass`, the vault file and keys.

9. Idempotency, cost control and clean-up

9.1 Idempotency

Running `ansible-playbook playbooks/site.yml` a second time reports mostly `ok` instead of `changed`, showing that the playbooks only change what is different.

Terminal: PLAY RECAP of the full ansible-playbook playbooks/site.yml run (failed=0, unreachable=0 for both servers).

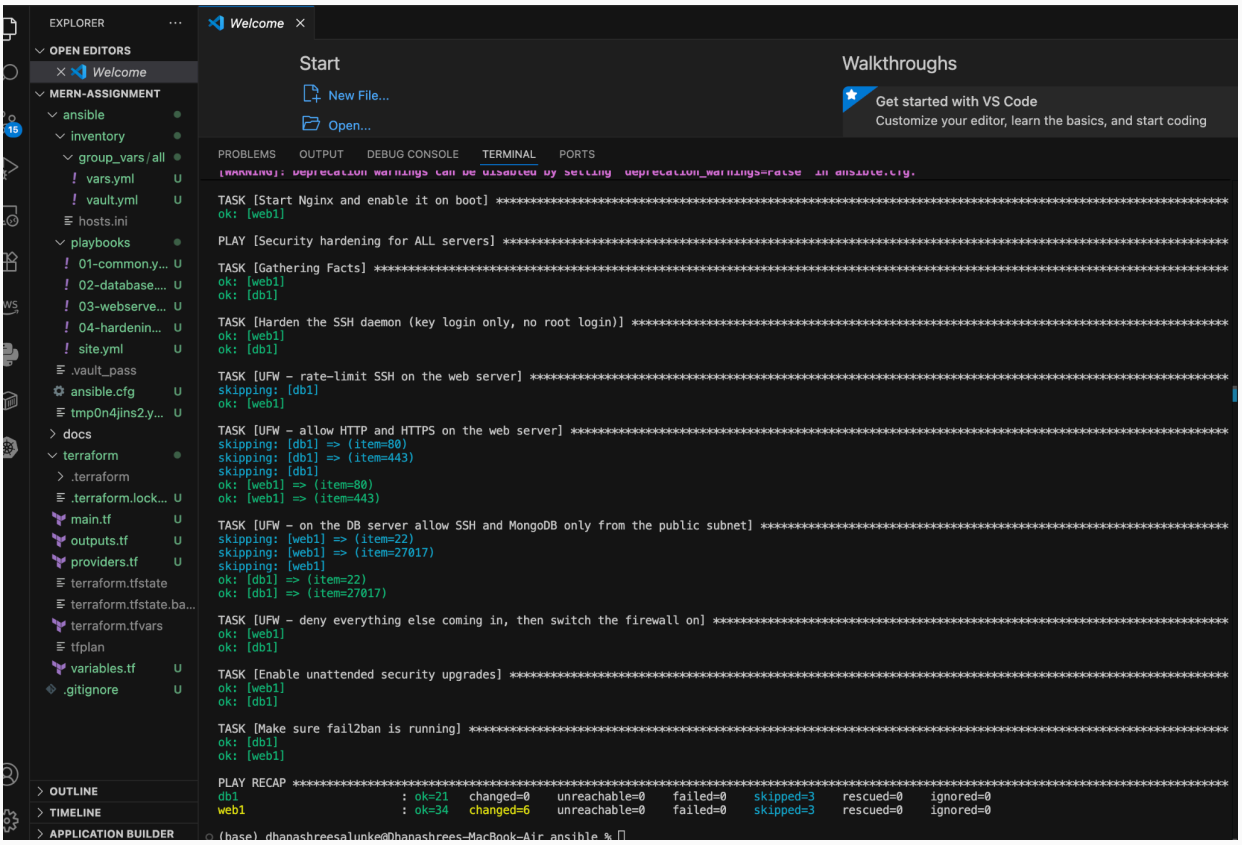


Figure 46: *Full playbook run*

9.2 Cost and clean-up

The NAT Gateway, the two EC2 instances and the public IPv4 addresses are billed by the hour, so the infrastructure is removed with `terraform destroy` after the evidence has been collected and the code has been pushed.

Capture: Optional (do this last): terraform destroy finishing with 'Destroy complete! Resources: 20 destroyed', and the EC2 console showing terminated instances.

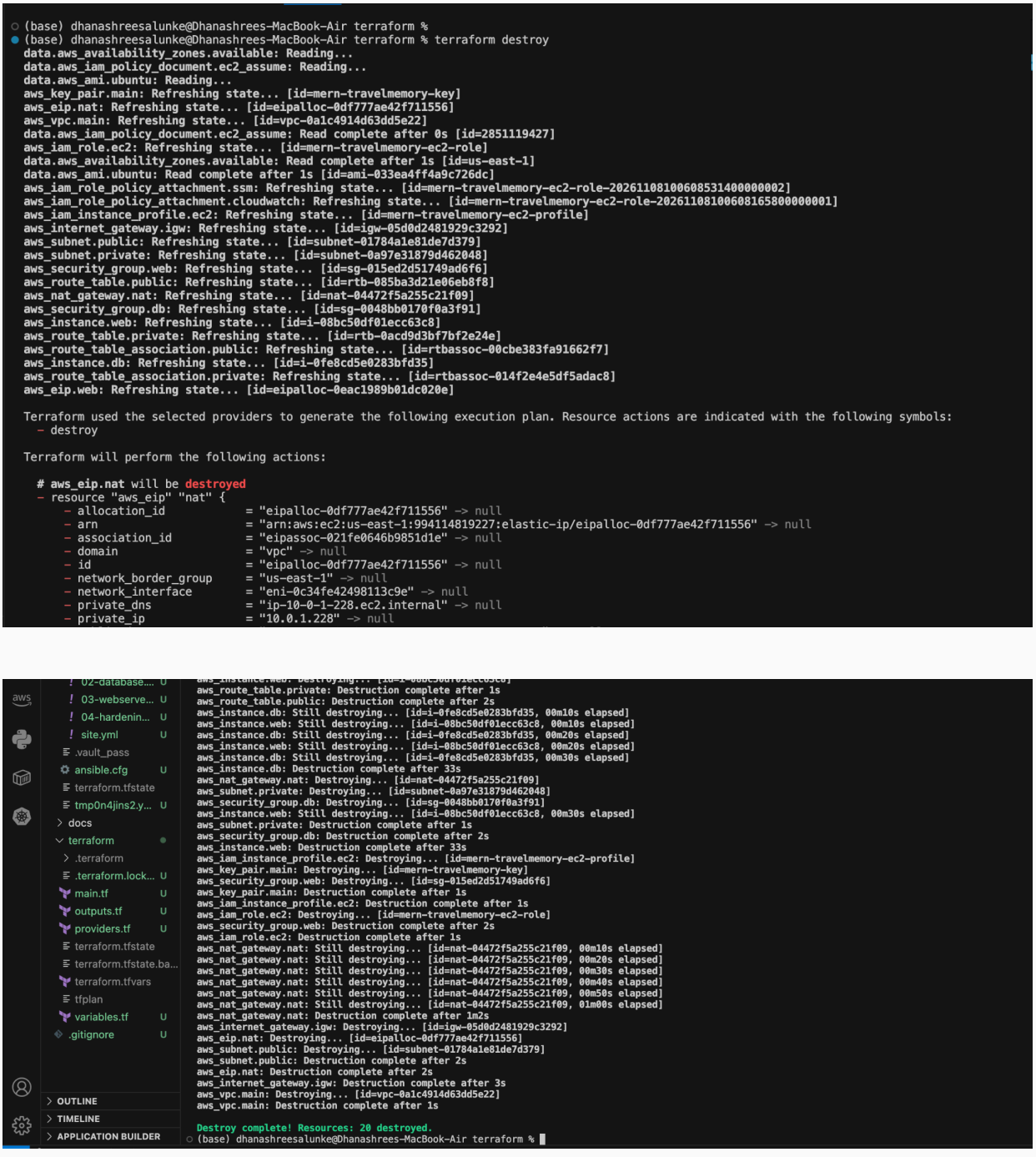


Figure 47: *Resources destroyed after the demonstration*

10. Conclusion and future improvements

10.1 Conclusion

EXPLANATION

I deployed the TravelMemory MERN application on AWS by creating the network, servers, security groups and IAM roles with Terraform, and by installing, configuring and securing the software with Ansible. The browser reaches the React frontend through Nginx, API calls on `/api` are forwarded to the Express backend on port 3001, and the backend talks to MongoDB on a private subnet that is reachable only from the web server. I learned that Terraform builds the infrastructure and Ansible configures it, that both can be re-run safely, and that secrets must be kept out of the repository. Troubleshooting problems such as the CORS setting and the Nginx upload limit taught me to check each layer’s logs in turn.

11. Deliverables checklist and repository details

Deliverable	Where it is	Done
Terraform scripts for AWS infrastructure	GitHub: `terraform/` (providers.tf, variables.tf, main.tf, outputs.tf)	[✅]
Ansible playbooks for configuration and deployment	GitHub: `ansible/` (ansible.cfg, inventory, group_vars, playbooks/01 to 04 and site.yml)	[✅]
Detailed report (this document)	GitHub: `docs/` folder	[✅]
Screenshots demonstrating the working application	Sections 4 to 7 of this report, also in `docs/screenshots/`	[✅]
Repository link submitted via Vlearn	Text/Word/PDF file with the GitHub URL	[✅]

11.1 Repository

Item	Value
GitHub repository URL	https://github.com/dhanacodes24/mern-terraform-ansible-assignment.git
Branch	main
Files excluded from Git	terraform.tfstate, terraform.tfvars, hosts.ini, .vault_pass, vault.yml, SSH keys
Submitted on Vlearn	09 Nov 2026

GitHub repository page in the browser showing the terraform/ and ansible/

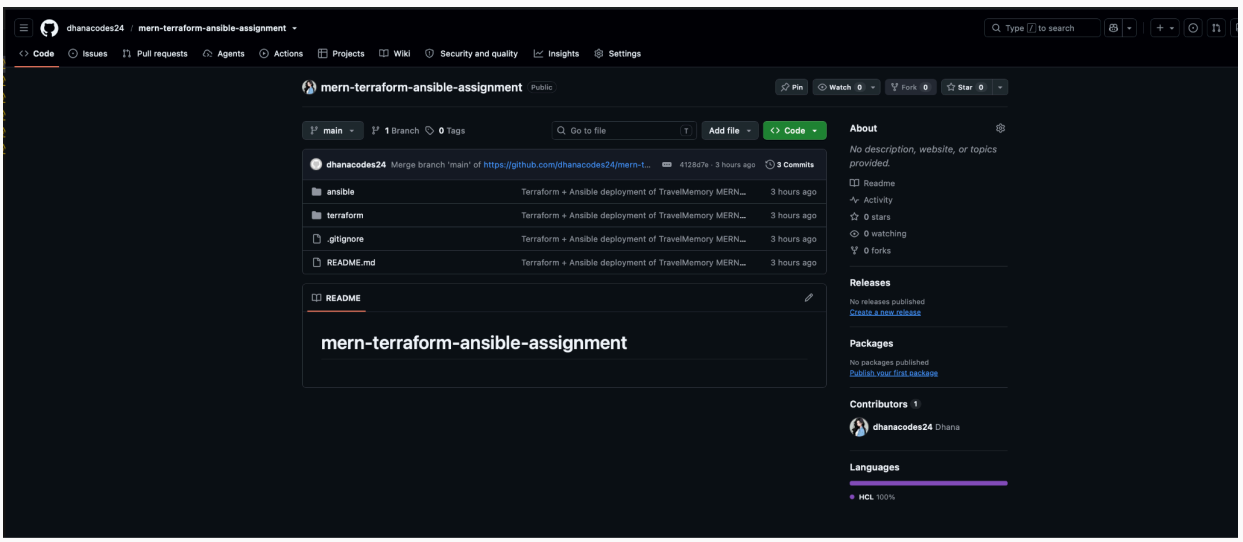


Figure 48: *GitHub repository containing the deliverables*

****END*****